VIETNAM NATIONAL UNIVERSITY, HO CHI MINH CITY
UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING



DEEP LEARNING AND ITS APPLICATIONS
(CO3133)

Assignment 1 Report

From Linear Models to Modern Sequence Models A Comparative Study for Image Classification

---

Instructor: Lê Thành Sách
Group: NNs
Project team
Nguyễn Nhật Nam – 2352778
Hồ Hồng Phúc Nguyên – 2352824
Phan Quốc Đại Sơn – 2353053

HO CHI MINH CITY, OCTOBER 2026

Abstract

This report compares five neural-network families on Fashion-MNIST: a linear classifier, multilayer perceptron, convolutional network, LSTM, and patch Transformer. A shared PyTorch pipeline uses a fixed Stratified Split of 54,000 training and 6,000 validation images, with 10,000 official test images. Models are trained from scratch with cross-entropy and Adam, and checkpoints are selected by validation macro-F1. Seven configurations, including three LSTM representations, are available at seed 42.

The CNN achieves the highest recorded test accuracy and macro-F1, 91.82% and 91.76%, with 50,378 parameters. Relative to the MLP, it makes 244 fewer errors, although its batched forward time is larger. Classwise results expose remaining garment confusions and a Dress recall advantage for the row LSTM. Rows lead the recurrent variants; patches reduce inference cost while lowering predictive quality. Learning curves, full-set confusion counts, structural operation estimates, and common-image predictions support the discussion of representation and inductive bias.

The results describe recorded configurations rather than causal architecture rankings. Single-seed evidence, unmatched regularization/capacity, absent original checkpoints, dirty-run traceability, and environment discrepancies limit stronger claims. Reproduction instructions and artifact fingerprints accompany the report.

Contents

Abstract **i**

I Problem and Data Description 1

1 Problem Formulation 1

1.1 Motivation . . . 1
1.2 Input and Output . . . 1
1.3 Research Questions . . . 1

2 Dataset Description 2

2.1 Dataset Source and License . . . 2
2.2 Dataset Statistics . . . 2
2.3 Exploratory Data Analysis . . . 3
2.3.1 Class Distribution . . . 3
2.3.2 Input Characteristics . . . 3
2.3.3 Class Imbalance Analysis . . . 4
2.3.4 Representative Samples . . . 4
2.3.5 Ambiguous Classes and Dataset Limitations . . . 4

3 Data Preparation 5

3.1 Train, Validation, and Test Split . . . 5
3.2 Leakage Prevention . . . 5
3.3 Preprocessing . . . 5
3.4 Data Augmentation . . . 6
3.5 Dataset and DataLoader . . . 6

II Methodology 7

4 Overall Pipeline 7

4.1 Pipeline Overview . . . 7
4.2 Training, Validation, and Test Loops . . . 8
4.3 Implementation and Library Usage . . . 8

5 Models 9

5.1 Linear Softmax Classifier . . . 9
5.1.1 Input Representation . . . 9

5.1.2 Architecture and Parameter Count 9
5.1.3 Strengths and Limitations 10
5.2 Multilayer Perceptron 10
5.2.1 Input Representation 10
5.2.2 Architecture and Activation Functions 10
5.2.3 Regularization and Parameter Count 10
5.2.4 Strengths and Limitations 11
5.3 Convolutional Neural Network 11
5.3.1 Architecture Design 11
5.3.2 Convolution Pooling and Feature Maps 11
5.3.3 Classification Head and Parameter Count 12
5.3.4 Strengths and Limitations 12
5.4 Long Short-Term Memory Network (LSTM) 12
5.4.1 Image to Sequence Representation 12
5.4.2 Timestep Definition and Input Dimensions 13
5.4.3 Recurrent Architecture and Hidden Representation 13
5.4.4 Classification Head and Parameter Count 13
5.4.5 Strengths and Limitations 13
5.5 Transformer 14
5.5.1 Tokenization and Input Representation 14
5.5.2 Token Embedding and Positional Encoding 14
5.5.3 Attention Inputs and Outputs 14
5.5.4 Encoder and Classification Head 15
5.5.5 Parameter Count, Strengths, and Limitations 15

6 Loss Function and Optimization **15**
6.1 Cross-Entropy Loss 15
6.2 Optimizer and Learning Rate Schedule 16
6.3 Regularization 16

7 Training and Evaluation Protocol **17**
7.1 Training Configuration 17
7.2 Hardware and Software Environment 17
7.3 Hyperparameter Selection 18
7.4 Checkpoint Selection 18
7.5 Evaluation Metrics 18
7.5.1 Accuracy 19

7.5.2 Macro-F1 . . . 19
7.5.3 Parameter Count . . . 19
7.5.4 Training Time . . . 19
7.5.5 Inference Time . . . 19
7.6 Fair Comparison Protocol . . . 20
7.7 Experimental Hypotheses and Controlled Factors . . . 20

III Implementation Results **22**

8 Training Behaviour **22**
8.1 Training and Validation Curves . . . 22
8.2 Convergence and Checkpoint Selection . . . 23
8.3 Overfitting and Underfitting Analysis . . . 24
8.4 Implementation Issues and Fixes . . . 25

9 Quantitative Results **25**
9.1 Main Model Comparison . . . 25
9.2 Classwise Performance . . . 26
9.3 Confusion Matrices . . . 27
9.4 Variation Across Random Seeds . . . 29

10 Qualitative Results **29**
10.1 Correct Predictions . . . 29
10.2 Difficult but Correct Cases . . . 30
10.3 Incorrect Predictions . . . 30

11 Model Comparison and Discussion **32**
11.1 Performance Differences . . . 32
11.2 Accuracy, Speed, and Parameter Trade-offs . . . 33
11.3 Effect of Input Representation . . . 34
11.4 Effect of Inductive Bias . . . 35
11.5 Answers to Research Questions . . . 37

12 Error Analysis **38**
12.1 Error Taxonomy . . . 38
12.2 Error Frequency and Common Class Confusions . . . 39
12.3 Representative Failure Cases . . . 39
12.4 Hypothesized Causes and Improvement Ideas . . . 40

13 Optional Ablation and Extensions **40**

13.1 Hypothesis and Experimental Design 40

13.2 Results 41

13.3 Discussion 41

14 Limitations **41**

14.1 Data Limitations 41

14.2 Method and Experimental Limitations 42

14.3 Generalization 42

15 Conclusion **42**

15.1 Main Findings 42

15.2 Future Work 43

16 Reproducibility **43**

16.1 Repository and Commit Reference 43

16.2 Environment and Dependencies 44

16.3 Dataset Preparation and Split Reconstruction 44

16.4 Training and Evaluation Commands 44

16.5 Checkpoint Access 45

16.6 Experiment Traceability 45

AI Usage Disclosure **46**

References **47**

A Complete Hyperparameter Table **48**

B Detailed Architecture Tables **48**

C Additional Curves and Confusion Matrices **49**

D Member Contributions **50**

Part I

Problem and Data Description

1 Problem Formulation

1.1 Motivation

Recognizing the category of a fashion item from an image is relevant to organizing product catalogs and supporting product search. This report studies a simplified version of this task using Fashion-MNIST. The objective is to compare how different neural network architectures learn from the same images within a reproducible experimental pipeline.

The comparison covers five model families: a linear classifier, a multilayer perceptron (MLP), a convolutional neural network (CNN), a long short-term memory network (LSTM), and a Transformer encoder. These models process the image in different ways: flattened pixels, a two-dimensional spatial grid, or a sequence of rows, columns, or patches. This makes the task suitable for examining how input representation and architectural assumptions relate to classification performance and computational cost.

The study is limited to the Fashion-MNIST benchmark. Its results should not be interpreted as evidence of performance on full-resolution product photographs, cluttered scenes, or production catalog systems.

1.2 Input and Output

Each sample is a grayscale image of size 28×28 pixels paired with one of ten class labels. The prediction unit is one image, and the task is single-label multiclass classification. After preprocessing, a batch of B images is represented by a tensor $X \in \mathbb{R}^{B\times 1\times 28\times 28}$.

For an individual image x, a model with parameters θ produces ten logits:

$$f_\theta : \mathbb{R}^{1\times 28\times 28} \to \mathbb{R}^{10}, \qquad z = f_\theta(x). \tag{1}$$

The predicted class is

$$\hat{y} = \underset{k\in\{0,\dots,9\}}{\arg\max}\, z_k. \tag{2}$$

All models return logits and are trained using cross-entropy loss. Softmax is not applied before the loss. Although their internal input representations differ, every model follows the same external input and output interface.

The main difficulties are distinguishing visually similar categories from low-resolution images and learning representations that generalize beyond the training set. The evaluation therefore considers both aggregate scores and the classes and examples on which each model makes errors.

1.3 Research Questions

The main research question is: *How do the five architecture families compare on Fashion-MNIST in terms of predictive quality and computational cost under a shared data split and evaluation protocol?*

We examine the following supporting questions:

1. How much do nonlinear models improve over the linear baseline in accuracy and macro-F1?
2. How do spatial processing in the CNN and sequence processing in the LSTM and Transformer relate to performance and class confusions?
3. How does representing an image as rows, columns, or patches affect the LSTM in the available experiments?
4. What trade-offs arise between accuracy, macro-F1, parameter count, training time, and inference time?

All main models use the same train, validation, and test partitions and evaluation metrics. Differences in model capacity, regularization, and training behavior are documented when interpreting the results; a performance difference alone does not isolate the causal effect of architecture. Numerical answers and supporting examples are presented in the results and discussion sections.

2 Dataset Description

2.1 Dataset Source and License

Fashion-MNIST contains fashion-product images released by Zalando Research and is the dataset used for the main comparison in this assignment [1]. The project loads it through `torchvision.datasets.FashionMNIST`; the download procedure is provided in `scripts/prepare_data.py`. The upstream repository distributes the dataset under the MIT license [2]. We use the official image and label files without adding or relabeling samples. MNIST is reserved for development and debugging, and no CIFAR-10 experiments are included in the committed A1 comparison.

To identify the dataset release reproducibly, we verified the MD5 checksums of all four local compressed files against the values published by the maintainers. All four matched. This identifies the data used more precisely than a download date alone.

2.2 Dataset Statistics

Table 1 summarizes the 70,000 Fashion-MNIST images and the partitions used in this project.

Table 1: Fashion-MNIST characteristics and partitions used in this project.

Property	Value
Prediction unit	One image with one class label
Classes	10, encoded by integers from 0 to 9
Image size and channels	28×28 pixels, one grayscale channel
Raw pixel representation	Unsigned 8-bit integers, intensity range $[0, 255]$
Official partitions	60,000 training images; 10,000 test images
Project partitions	54,000 train; 6,000 validation; 10,000 test
Training/validation split	Stratified Split, 90%/10% per class, seed 42
Storage format	IDX image and label files, distributed as gzip archives
Compressed download size	30,878,645 bytes (approximately 30.88 MB)
Uncompressed IDX size	54,950,048 bytes (approximately 54.95 MB)

We apply a *Stratified Split* to the 60,000-image official training partition, allocating 90% of each class to training and 10% to validation with random seed 42. This yields 54,000 training images and 6,000 validation images while preserving class proportions. The official test partition of 10,000 images remains unchanged. Implementation details are described in the next section.

The storage sizes above were measured from the four local archives and their four extracted IDX files, respectively. They exclude checkpoints, plots, and other experiment artifacts. Each image contributes 784 pixel values, and the annotations contain class IDs rather than bounding boxes or segmentation masks.

2.3 Exploratory Data Analysis

The EDA examines class counts, image dimensions, pixel representation, and examples from each category. We checked the saved split indices against the raw label files and confirmed that the counts in `results/a1/tables/a1_class_distribution.csv` match the data. The figures below reuse the existing EDA artifacts in `results/a1/figures/`.

2.3.1 Class Distribution

Figure 1 shows the training and validation counts for all ten classes. Each class has 6,000 images in the official training partition. The Stratified Split retains 5,400 per class for training and 600 per class for validation; the official test set contains 1,000 per class.

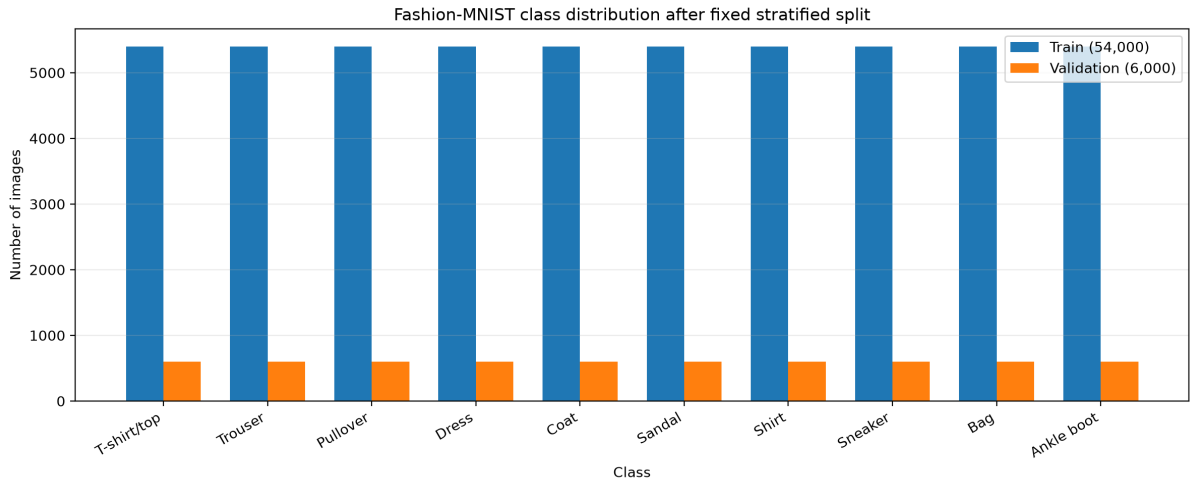


Figure 1: Class distribution after the fixed Stratified Split. All classes have identical support within each partition.

2.3.2 Input Characteristics

All images have the same dimensions, so the comparison does not require resizing or padding to accommodate variable image sizes. Inspection of Figure 2 shows light and dark garments, differences in silhouette, and varying contrast against the background. Fine details are difficult to distinguish at this resolution.

After conversion to tensors, an image has shape $[1, 28, 28]$ and a batch has shape $[B, 1, 28, 28]$. The training configuration records a pixel mean of 0.286209 and a standard deviation of 0.353160 on the $[0, 1]$ scale. The repository's statistics script computes these values using only the fixed 54,000-image training subset. Their use in normalization is explained in the data-preparation section.

2.3.3 Class Imbalance Analysis

Every class accounts for exactly 10% of each partition. The ratio of the largest to the smallest class count is therefore 1.0, indicating no class-frequency imbalance. Class reweighting or oversampling is not motivated by these counts. This balance does not imply equal classification difficulty: classes with similar appearances may still have different error rates. We report macro-F1 alongside accuracy to assess classwise precision and recall, and use confusion matrices to locate the remaining differences.

2.3.4 Representative Samples

Figure 2 shows one existing EDA example per class from the training subset. The grid illustrates the label mapping and visible differences between categories. It is illustrative rather than an exhaustive account of within-class variation.

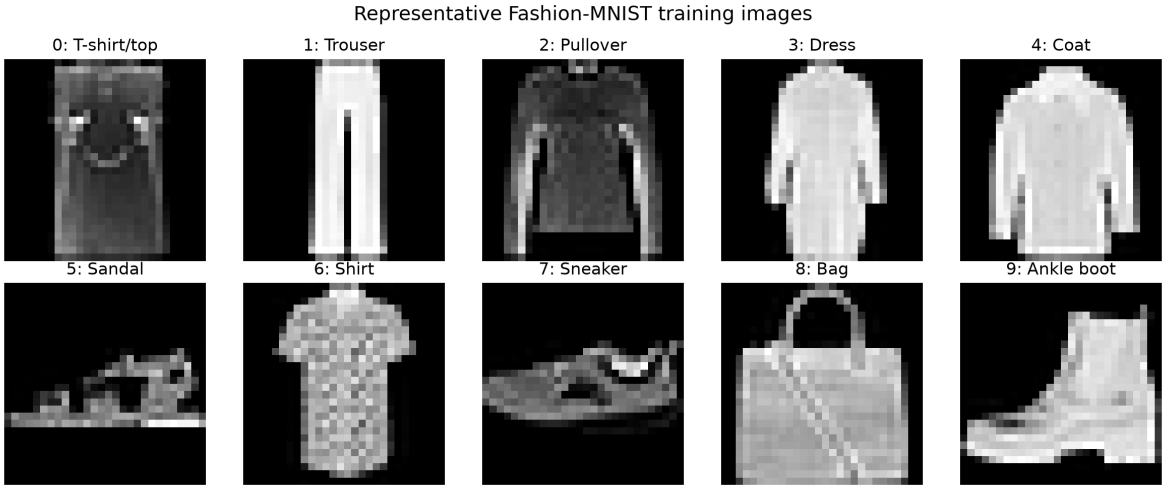


Figure 2: One Fashion-MNIST training example per class, labeled with its class ID and name.

2.3.5 Ambiguous Classes and Dataset Limitations

Upper-body categories such as T-shirt/top, shirt, pullover, and coat can share similar outlines. At 28×28 pixels, sleeve shape, garment length, and coarse intensity patterns may carry more visible information than small details. This motivates examining these categories in the later error analysis; it does not establish which class pairs the trained models confuse most often.

The benchmark has several limits. Grayscale images remove color cues, and the small spatial resolution restricts the available texture and shape detail. The examples depict isolated fashion items, so they do not represent the full variation of backgrounds, occlusions, viewpoints, and lighting in everyday photographs. A balanced ten-class benchmark also does not reproduce the potentially uneven category frequencies of a deployed application. Consequently, the experiments support conclusions about this benchmark and the recorded training protocol; deployment performance and broader domain generalization remain untested.

3 Data Preparation

This section describes how the official Fashion-MNIST records become the image tensors used by all five model families. The preparation procedure is shared across models; architecture-specific flattening or conversion to sequences takes place inside the models.

3.1 Train, Validation, and Test Split

We apply a *Stratified Split* to the official 60,000-image training partition using `StratifiedShuffleSplit` in `src/data/split.py`. The splitter uses one split, a validation size of 6,000, and random seed 42. It assigns 90% of each class to training and 10% to validation, giving 54,000 training images and 6,000 validation images. The official 10,000-image test partition is retained separately.

The unit of splitting is one image index. The selected indices are stored in `results/a1/splits/split_seed42.json`, together with the seed, partition sizes, and class counts. Every main model reads this same file rather than drawing a new split at the start of training. Consequently, the models are compared on identical examples, and differences in random partition selection do not enter the main comparison.

3.2 Leakage Prevention

The training and validation index sets are disjoint and together cover all 60,000 official training images. We verified these properties directly from the saved indices. The dataset wrapper uses the official training partition for both subsets and the separate official test partition for testing.

Normalization statistics are computed from the 54,000 training images only and then reused for validation and testing. The training entry point passes only the training and validation loaders to the fitting routine, and the best checkpoint is selected using validation macro-F1. Test predictions are produced by the separate evaluation entry point after loading that checkpoint. This separates the implemented fitting and checkpoint-selection steps from test evaluation.

These checks establish separation at the image-index level. They do not establish that visually similar or near-duplicate products are absent across partitions; duplicate-aware grouping is not implemented in the current pipeline.

3.3 Preprocessing

The pipeline retains the official images and labels without filtering or relabeling records. Since every image already has one grayscale channel and a resolution of 28×28, no resizing or padding is applied. Preprocessing consists of tensor conversion followed by channelwise normalization:

1. **Tensor conversion.** `ToTensor()` converts an image from unsigned 8-bit pixels to a floating-point tensor of shape $[1, 28, 28]$. A raw pixel value $p \in \{0, \ldots, 255\}$ becomes $u = p/255$.
2. **Normalization.** `Normalize` applies the same training-derived constants to every image:

$$x = \frac{u - \mu_{\text{train}}}{\sigma_{\text{train}}}, \qquad \mu_{\text{train}} = 0.286209, \quad \sigma_{\text{train}} = 0.353160. \tag{3}$$

The statistics script computes the mean and population standard deviation across all pixels in the fixed training subset, rather than separately for each image. A local recomputation matched

the configured constants to six decimal places. Normalized values are not constrained to $[0, 1]$: the theoretical interval is approximately $[-0.8104, 2.0212]$. Labels retain their integer class IDs from 0 to 9.

3.4 Data Augmentation

The reported main configuration sets `augmentation: none`. No random crops, flips, rotations, or other synthetic image transformations are applied. Training, validation, and testing therefore share the same deterministic preprocessing. The current transform builder rejects unsupported training-augmentation settings instead of silently ignoring them. This provides a common input pipeline for the architecture comparison; the effect of augmentation has not been evaluated in the committed main experiments.

3.5 Dataset and DataLoader

`FashionMNISTSubset` wraps the official dataset and maps each subset index to its saved original image index. Its `__getitem__` method applies preprocessing and returns an image tensor and an integer label. The loader factory returns separate training, validation, and test loaders with a configured batch size of 128 and two worker processes. Training samples are shuffled, while validation and test samples retain their stored order. The final incomplete batch is retained.

The loaders use seeded PyTorch generators: 42 for training, 43 for validation, and 44 for testing. The worker initialization function seeds NumPy and Python's random module from each worker's PyTorch seed. Pinned host memory is enabled when CUDA is available. These settings support reproducible sample loading, although identical seeds alone do not guarantee identical numerical results across all devices and library versions.

For a concrete example, we inspected the first validation batch using the repository's dataset wrapper and transform builder. The resulting tensors were:

```
images: shape [128, 1, 28, 28], dtype torch.float32
labels: shape [128],            dtype torch.int64
pixel range: [-0.810423, 2.021155]
first 8 labels: [3, 1, 9, 8, 2, 1, 2, 9]
```

This local inspection used zero worker processes while preserving the configured batch size and validation order; the experiment configuration remains at two workers. The batch was checked against direct pixel scaling and normalization. Its metadata are saved in `results/a1/tables/a1_preprocessing_example.json`. The inspection verifies data preparation only and does not constitute a new training run.

Part II

Methodology

4 Overall Pipeline

All five model families use a shared PyTorch training and evaluation pipeline. The selected model changes the mapping from an image batch to class logits; the saved data partitions, preprocessing, loss interface, checkpoint rule, and evaluation outputs remain common. This organization supports comparisons in which results can be traced to a specific configuration and run directory.

4.1 Pipeline Overview

Figure 3 summarizes the data flow. The preparation stage reads the official Fashion-MNIST files, applies the saved Stratified Split, and constructs the loaders described in Section 3. Each model receives images of shape $[B, 1, 28, 28]$ and returns logits of shape $[B, 10]$. Flattening, spatial feature extraction, or sequence construction is handled within the model, so the training engine does not need a different loop for each architecture.

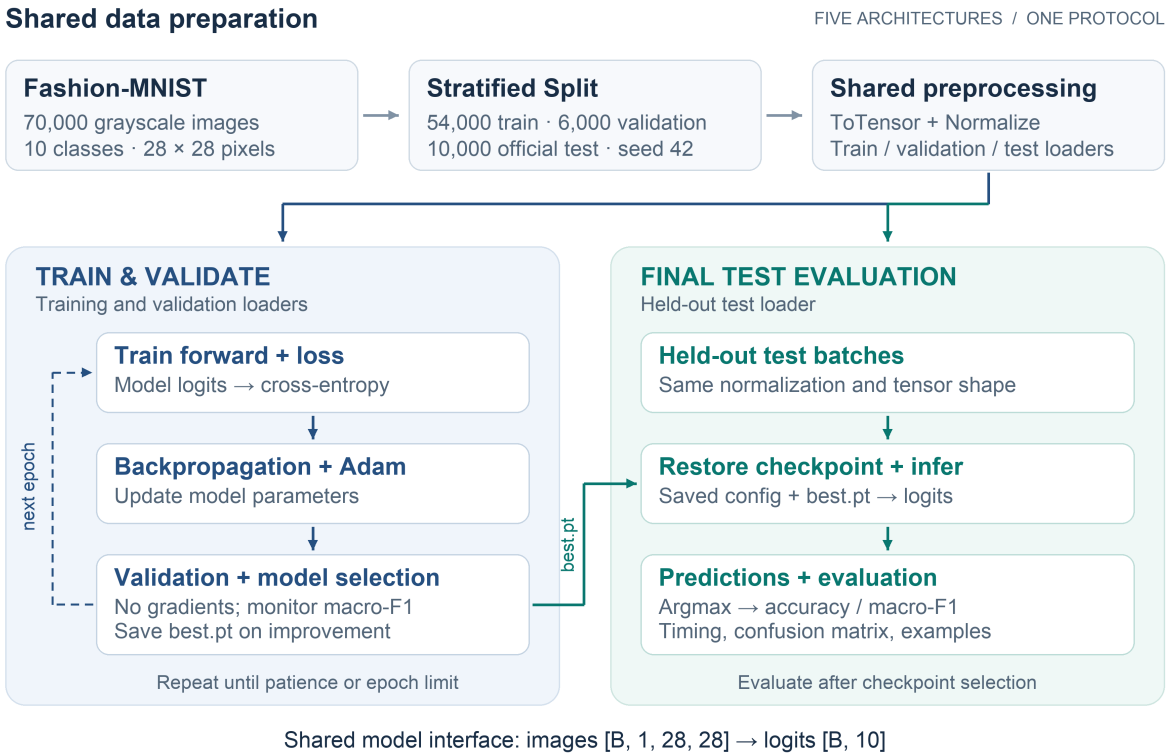


Figure 3: Shared training and evaluation pipeline. The training and validation loop selects the best checkpoint, which is transferred to held-out test evaluation.

The training entry point, `src/train.py`, reads `configs/a1.yaml`, sets the random seeds, constructs the selected model through the model registry, and creates a run directory under `results/a1/runs/`. It saves a copy of the configuration before fitting. The run also records training history, parameter count, elapsed training time, environment information, and the

source-code commit identifier.

The evaluation entry point, `src/evaluate.py`, takes an explicit run directory. It reconstructs the model using that run's saved configuration and loads its selected weights. Evaluation therefore uses the configuration associated with the checkpoint, even if the shared configuration file has subsequently changed.

4.2 Training, Validation, and Test Loops

Training. At the start of each epoch, the engine calls `model.train()`. Each batch is transferred to the selected device, accumulated gradients are cleared, and a forward pass produces logits. Cross-entropy compares those logits with the integer labels. Backpropagation computes parameter gradients, and the configured optimizer performs an update. Adam is used in the main configuration. Training loss is accumulated as a sum weighted by batch size and divided by the number of processed images, so the final smaller batch is weighted according to its sample count.

Validation and checkpoint selection. After each training epoch, the engine switches to `model.eval()` and evaluates the validation loader inside `torch.no_grad()`. Model parameters are not updated. The loop aggregates sample-weighted loss and collects all predictions and labels before computing accuracy and macro-F1. Whenever validation macro-F1 strictly improves, the model state is saved as `best.pt`. Training stops after the configured patience is exhausted or the epoch limit is reached. The sequence of training loss and validation metrics is written to `history.csv`.

Testing and post-processing. The test loop uses the same evaluation function after the selected checkpoint has been restored. The predicted label is the index of the largest logit; no additional thresholding or class-specific post-processing is applied. The script writes accuracy, macro-F1, a confusion matrix, and inference-timing metadata to `test_metrics.json`. It also produces training curves, a confusion-matrix plot, and grids of correct and incorrect predictions.

The timing routine measures forward passes after loading the test tensors onto the target device. It excludes data loading, normalization, and host-to-device transfer, and reports the median duration over repeated passes. Consequently, this quantity describes model inference cost under the recorded protocol rather than complete application response time.

4.3 Implementation and Library Usage

Project-specific implementation. The repository defines the saved split procedure, dataset wrapper, transform configuration, model architectures, image-to-sequence conversion, training and validation loops, checkpoint-selection logic, run metadata, and reporting scripts. Model-specific settings are passed through the common registry in `src/models/__init__.py`. These components provide the experimental structure of the comparison.

Library components. PyTorch supplies tensors, automatic differentiation, optimization, data loading, and standard neural-network layers, including the recurrent and Transformer encoder modules. Torchvision supplies the Fashion-MNIST reader and image transforms. Scikit-learn supplies `StratifiedShuffleSplit`, macro-F1, and the confusion matrix. NumPy and pandas support aggregation and tabular outputs, Matplotlib generates the plots, and PyYAML reads and writes configurations. The group composes these building blocks and implements the surrounding pipeline; the report distinguishes that work from the library-provided operations.

5 Models

The comparison uses five architecture families: a linear classifier, an MLP, a CNN, an LSTM, and a Transformer encoder. All receive the normalized image tensor defined in Section 3 and produce ten logits for cross-entropy training. The architectures are constructed from standard PyTorch components and trained without pretrained weights.

The descriptions below follow the configurations saved with the reported runs, rather than relying only on constructor defaults. We instantiated all seven saved configurations, including the three LSTM input representations, checked their output shape on a two-image batch, and matched their parameter counts to the recorded run metadata. This inspection did not load trained checkpoints or produce new predictive results. The checks are recorded in `results/a1/tables/a1_model_verification.json`. Parameter counts include trainable weights and biases; non-trainable buffers such as BatchNorm running statistics are excluded.

The architecture illustrations show the configured layer dimensions. Image thumbnails use one example from the official training partition; neuron nodes and feature-map grids are schematic and do not depict measured activations.

5.1 Linear Softmax Classifier

5.1.1 Input Representation

The linear baseline flattens each image from $[1, 28, 28]$ to a vector $x \in \mathbb{R}^{784}$. Pixel positions retain a fixed order, but the model receives no explicit two-dimensional neighborhood structure. Its purpose is to establish the performance of a classifier with a single affine mapping from pixels to classes. Figure 4 illustrates this mapping.

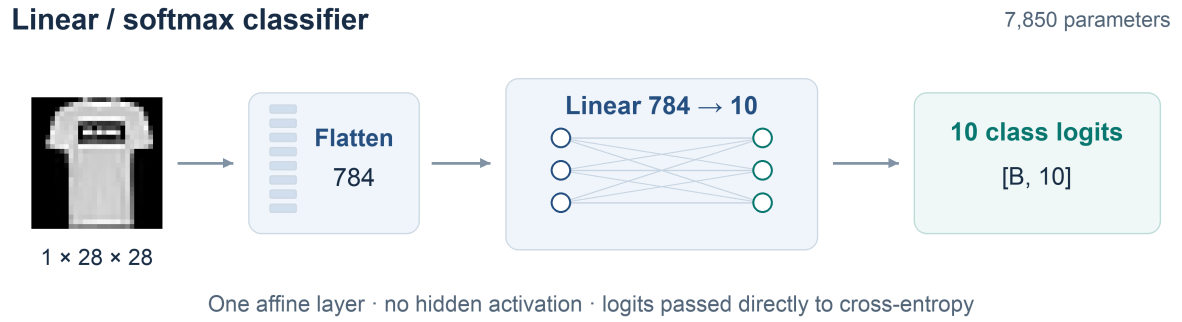


Figure 4: Linear classifier: flatten 784 pixels and apply one affine layer to obtain ten logits. Only selected connections are drawn.

5.1.2 Architecture and Parameter Count

The implementation in `src/models/linear.py` consists of `Flatten` followed by one linear layer:

$$z = Wx + b, \qquad W \in \mathbb{R}^{10\times 784}, \quad b \in \mathbb{R}^{10}. \tag{4}$$

It contains $784 \times 10 + 10 = 7{,}850$ trainable parameters. The term softmax classifier refers to the multiclass probabilistic interpretation of its logits. The forward method returns the logits directly, and `CrossEntropyLoss` handles the corresponding normalization during loss computation. No hidden activation or explicit softmax layer is appended.

5.1.3 Strengths and Limitations

The model is compact and provides a simple reference for the added capacity of the other architectures. Each class score is a weighted sum of the pixels, which makes the mapping straightforward to inspect. Its affine decision boundaries cannot directly express nonlinear combinations of image features, and its position-specific weights do not share information across translated local patterns. These restrictions motivate the nonlinear and spatial models below; their practical effects are assessed using the recorded experiments.

5.2 Multilayer Perceptron

5.2.1 Input Representation

The MLP uses the same flattened 784-dimensional image vector as the linear baseline. This keeps the input representation fixed while adding hidden nonlinear transformations. The network is implemented in `src/models/mlp.py` and illustrated in Figure 5.

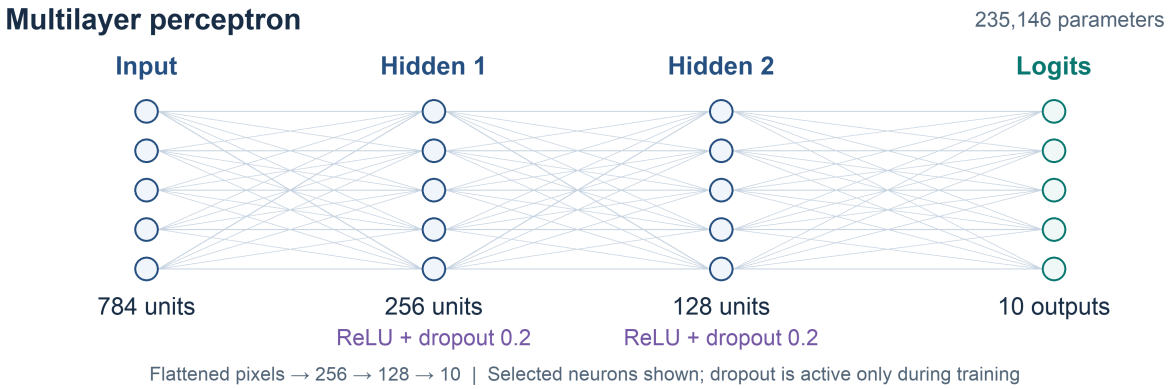


Figure 5: MLP with 256- and 128-unit hidden layers, each followed by ReLU and dropout. The drawing shows selected neurons rather than the full layer widths.

5.2.2 Architecture and Activation Functions

The saved configuration specifies two hidden layers with 256 and 128 units. Each hidden linear layer is followed by ReLU and dropout; the final layer maps 128 features to ten logits. The forward computation is

$$h_1 = D_{0.2}(\mathrm{ReLU}(W_1 x + b_1)), \tag{5}$$
$$h_2 = D_{0.2}(\mathrm{ReLU}(W_2 h_1 + b_2)), \tag{6}$$
$$z = W_3 h_2 + b_3, \tag{7}$$

where $D_{0.2}$ denotes dropout with probability 0.2 during training. ReLU applies $\max(0, a)$ elementwise, allowing the composition to represent nonlinear decision boundaries. The output layer has no activation because the shared loss expects logits.

5.2.3 Regularization and Parameter Count

Dropout is applied after both hidden activations and is disabled by `model.eval()` during validation and testing. It randomly removes hidden activations during training, providing regular-

ization without adding trainable parameters. The three affine layers contribute

$$(784 \times 256 + 256) + (256 \times 128 + 128) + (128 \times 10 + 10) = 235{,}146 \tag{8}$$

parameters. No BatchNorm layer is used in this MLP.

5.2.4 Strengths and Limitations

The hidden layers can learn combinations of pixels that the linear baseline cannot express. However, fully connected layers assign separate weights to each spatial position. The network therefore has no built-in local weight sharing and uses substantially more parameters than the configured CNN. Its performance also depends on hidden width and regularization, so an MLP–linear comparison reflects both nonlinear capacity and a change in parameter count.

5.3 Convolutional Neural Network

5.3.1 Architecture Design

`SmallCNN`, defined in `src/models/cnn.py`, preserves the image's spatial layout. The configured feature extractor contains two blocks, with 32 and 64 output channels. Each block applies a 3×3 convolution with stride 1 and padding 1, BatchNorm, ReLU, and 2×2 max pooling with stride 2. The convolution preserves the spatial dimensions, while pooling halves them. Figure 6 shows the blocks and resulting feature dimensions.

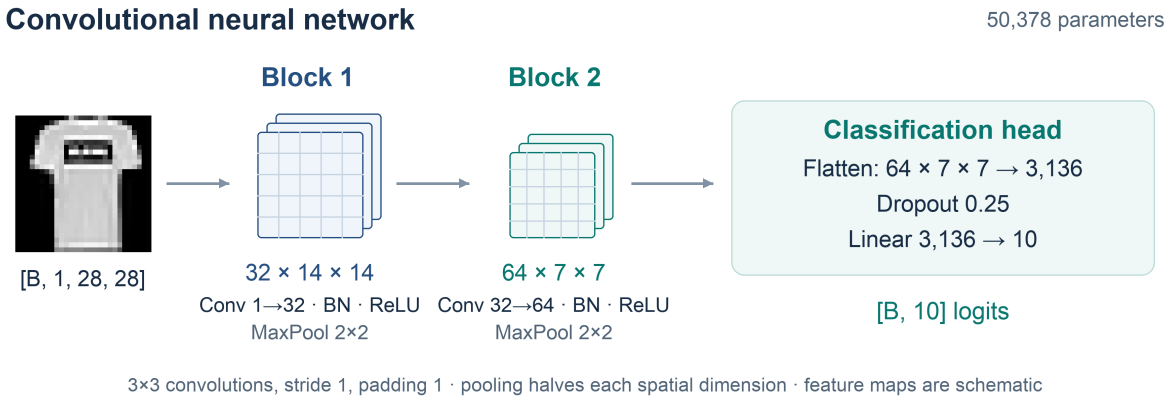


Figure 6: CNN feature extraction and classification. Feature-map stacks are schematic; the annotated dimensions are taken from the configured model. BN denotes BatchNorm.

5.3.2 Convolution Pooling and Feature Maps

A convolution applies shared local filters across the image. Each output channel forms a feature map describing the responses of one learned filter at different positions. This encourages reuse of local patterns such as boundaries and shapes. ReLU introduces nonlinearity, and max pooling summarizes the strongest response in each 2×2 neighborhood while reducing spatial resolution. BatchNorm has learned scale and shift parameters and maintains running statistics for evaluation.

These operations introduce a spatial inductive bias that is absent from the flattened linear and MLP inputs. Pooling reduces the size of the representation, but it also discards some positional detail. The architecture does not guarantee complete translation invariance.

5.3.3 Classification Head and Parameter Count

The final feature tensor contains $64 \times 7 \times 7 = 3{,}136$ values per image. The classification head flattens this tensor, applies dropout with probability 0.25, and uses a linear layer to produce ten logits. The feature extractor contributes 19,008 parameters: 18,816 convolutional weights and biases, plus 192 BatchNorm scale and shift parameters. The head contributes $3136 \times 10 + 10 = 31{,}370$, for a total of **50,378 parameters**. Dropout is present in the head in the implementation used for the saved run.

5.3.4 Strengths and Limitations

Local connectivity and parameter sharing allow this CNN to learn spatial features with fewer parameters than the configured MLP. Its two pooling stages also reduce the representation before classification. The shallow convolutional stack and downsampling may limit the detail that reaches the head, particularly for visually similar garments. These are architectural considerations, rather than claims that every class benefits equally; the confusion matrices and examples are used to examine the observed behavior.

5.4 Long Short-Term Memory Network (LSTM)

5.4.1 Image to Sequence Representation

The recurrent model treats an image as an ordered sequence rather than a two-dimensional convolutional input. The helper `to_sequence` in `src/models/rnn.py` supports rows, columns, and non-overlapping patches. All saved recurrent runs use an LSTM cell; the class also supports GRU, but no GRU result is included in the committed comparison.

The row representation is the main configured LSTM input. Column and patch representations are additional experiments using the same recurrent implementation. The image-to-sequence conversion changes only the arrangement of the input values; it is not a learned feature extractor.

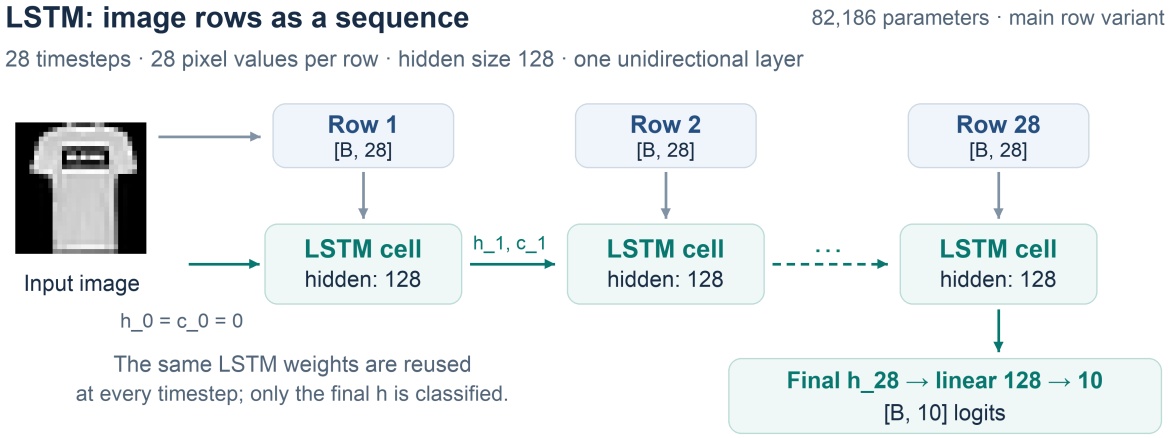


Figure 7: The main row-based LSTM, unrolled over selected timesteps. Recurrent weights are shared, and the final hidden state feeds the classifier. Intermediate timesteps are omitted.

Figure 7 illustrates the row-based computation; the alternative sequence representations are detailed below.

5.4.2 Timestep Definition and Input Dimensions

For rows, timestep t contains the 28 pixels in image row t, giving 28 timesteps with 28 features each. The column representation transposes the image so that each timestep contains one column. The patch representation partitions the image into a 4×4 grid of 7×7 patches, flattened and ordered across the grid. Table 2 summarizes the three variants.

Table 2: LSTM input representations and verified parameter counts.

Representation	Timesteps	Features	Sequence shape	Parameters
Rows	28	28	$[B, 28, 28]$	82,186
Columns	28	28	$[B, 28, 28]$	82,186
7×7 patches	16	49	$[B, 16, 49]$	92,938

5.4.3 Recurrent Architecture and Hidden Representation

The model uses one unidirectional LSTM layer with hidden size $H = 128$ and batch-first input. Its hidden and cell states are initialized to zero by the PyTorch module because no initial states are supplied. At each timestep, the cell combines the current input with the previous states; its input, forget, and output gates control the memory update [3]. Both states have 128 features per image.

The final hidden state $h_T \in \mathbb{R}^{128}$ summarizes the sequence for classification. The implementation extracts this state from the LSTM's returned hidden-state tuple, rather than flattening all timestep outputs. No bidirectional processing, extra recurrent layer, or explicit dropout layer is used in these runs.

5.4.4 Classification Head and Parameter Count

A linear layer maps h_T to ten logits, contributing $128 \times 10 + 10 = 1{,}290$ parameters. For an input width F, the LSTM layer contains $4H(F + H + 2)$ parameters, accounting for its four gates, input and recurrent matrices, and two bias vectors per gate. Thus, the row and column variants contain

$$4 \times 128 \times (28 + 128 + 2) + 1{,}290 = 82{,}186 \tag{9}$$

parameters. Using 49 features per patch raises the total to 92,938. Therefore, rows versus columns keeps the parameter count fixed, whereas the patch comparison changes both sequence structure and input-projection size.

5.4.5 Strengths and Limitations

The recurrent state allows information from earlier rows, columns, or patches to influence the final prediction. However, the chosen scan order imposes an ordering on an otherwise spatial object, and classification depends on compressing the sequence into the final hidden state. Recurrent timesteps also have sequential dependencies. These properties motivate the representation experiments, whose measured quality and runtime are analyzed in the results section.

5.5 Transformer

5.5.1 Tokenization and Input Representation

The model in `src/models/transformer.py` uses a Transformer encoder to classify image patches. The saved configuration divides each image into 16 non-overlapping 7×7 patches, using the same patch-conversion helper as the LSTM. Flattening each patch gives a tensor of shape $[B, 16, 49]$. The model is trained from scratch and does not use a pretrained vision backbone. Figure 8 summarizes the tokenization, encoder stack, and classification head.

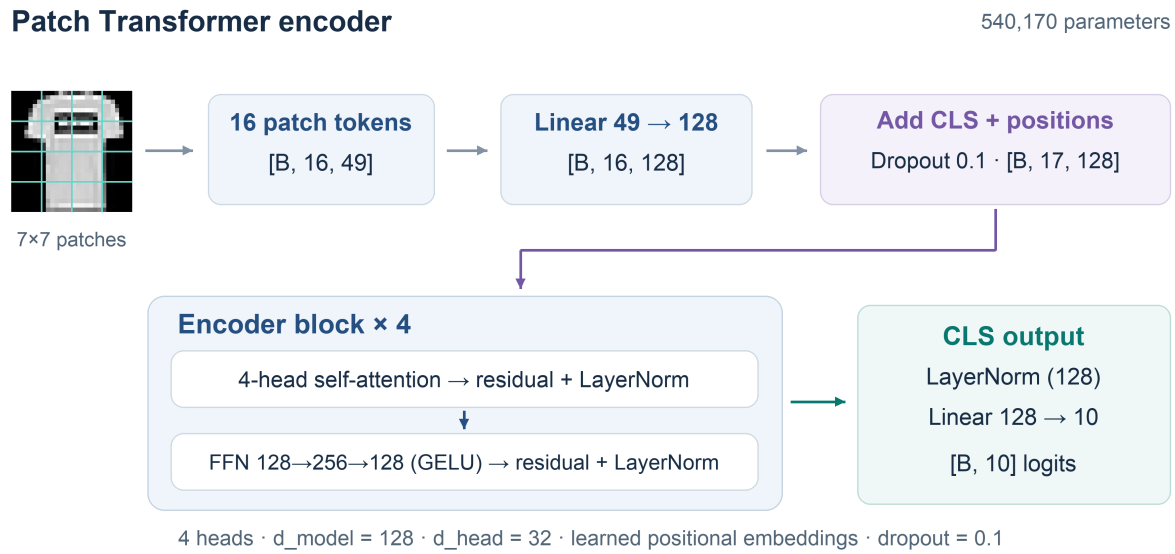


Figure 8: Patch Transformer: sixteen image patches become seventeen embedded tokens after adding CLS. Four encoder blocks process the tokens; the final CLS representation is normalized and classified.

5.5.2 Token Embedding and Positional Encoding

A learned linear projection maps each 49-dimensional patch to an embedding of width $d = 128$. A learned classification token, denoted CLS, is prepended to the 16 patch embeddings. The resulting sequence has 17 tokens. Learned positional embeddings specify their order, including the position of CLS:

$$E^{(0)} = D_{0.1}([e_{\text{CLS}}; e_1; \ldots; e_{16}] + P), \qquad P \in \mathbb{R}^{17\times 128}. \tag{10}$$

Here $e_i = W_e p_i + b_e$ is the projection of flattened patch p_i, and $D_{0.1}$ is dropout with probability 0.1. The encoder input therefore has shape $[B, 17, 128]$. Although sinusoidal positions are supported in the implementation, the reported Transformer run uses learned positions.

5.5.3 Attention Inputs and Outputs

The encoder uses four attention heads. Within a head, learned projections of the current token representations provide queries Q, keys K, and values V. Scaled dot-product attention computes [4]

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V, \qquad d_k = 128/4 = 32. \tag{11}$$

For one image, each head uses $Q, K, V \in \mathbb{R}^{17\times 32}$ and produces an output of shape $[17, 32]$. Concatenating the four head outputs and applying the output projection restores the token width to 128. Each token can therefore aggregate information from the full 17-token sequence. No causal or padding mask is passed: the task uses complete images with a fixed number of patches.

5.5.4 Encoder and Classification Head

The architecture stacks four `TransformerEncoderLayer` blocks. Each block contains multi-head self-attention and a positionwise feed-forward network with dimensions $128 \to 256 \to 128$ and a GELU activation. Residual connections, dropout with probability 0.1, and two LayerNorm modules are used in each block. The implementation retains PyTorch's post-normalization order, with normalization after the residual additions [5].

After the fourth block, the model selects the CLS output, applies a further LayerNorm, and maps its 128 features to ten logits. It does not average the patch-token outputs. The output shape is $[B, 10]$, matching the common interface.

5.5.5 Parameter Count, Strengths, and Limitations

The patch projection contributes 6,400 parameters, CLS contributes 128, and the learned positions contribute 2,176. Each encoder block contains 132,480 parameters, giving 529,920 across four blocks. The final LayerNorm and classifier contribute 256 and 1,290, respectively. The total is

$$6{,}400 + 128 + 2{,}176 + 529{,}920 + 256 + 1{,}290 = 540{,}170. \tag{12}$$

Self-attention combines distant patches without a recurrent scan. Patch projections and positional embeddings encode content and order without a convolutional feature extractor. As the largest configured model, its 540,170 parameters must be considered alongside predictive quality and runtime; global attention does not itself establish a performance advantage.

6 Loss Function and Optimization

All seven saved configurations use the same classification objective and optimizer settings. This section distinguishes these shared settings from the regularization built into each architecture. The description follows `src/engine/trainer.py` and the configuration snapshots stored with the reported runs.

6.1 Cross-Entropy Loss

For a batch of B images, every model returns logits $z \in \mathbb{R}^{B\times 10}$ and receives integer targets $y_i \in \{0, \ldots, 9\}$. The training objective is the mean multiclass cross-entropy:

$$\mathcal{L}_{\mathrm{CE}} = -\frac{1}{B}\sum_{i=1}^{B} \log \frac{\exp(z_{i,y_i})}{\sum_{k=0}^{9} \exp(z_{i,k})}. \tag{13}$$

The loss encourages the correct class to receive a larger logit relative to the competing classes. In the implementation, `build_loss` constructs `nn.CrossEntropyLoss()` with its default mean reduction, no class weights, and no label smoothing [6]. The model output is passed directly to

this loss; an explicit softmax is not applied beforehand. Class weighting is not used because the training classes have equal sample counts.

For epoch-level reporting, each batch's mean loss is multiplied by its actual batch size before accumulation. Dividing the accumulated sum by the total number of processed images gives a sample-weighted average, including the final incomplete batch. The same aggregation is used during validation. Cross-entropy supplies the differentiable training objective, while validation macro-F1 supplies the checkpoint-selection criterion; the code does not directly optimize macro-F1 through backpropagation.

6.2 Optimizer and Learning Rate Schedule

The main runs use `torch.optim.Adam` with a base learning rate $\alpha = 0.001$ and zero weight decay. The optimizer factory leaves Adam's remaining arguments at their defaults: $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$, and AMSGrad disabled [7]. Adam maintains exponential moving averages of the gradient and its elementwise square, then uses their bias-corrected estimates $\widehat{m}_t$ and $\widehat{v}_t$ to update the parameters:

$$\theta_t = \theta_{t-1} - \alpha \frac{\widehat{m}_t}{\sqrt{\widehat{v}_t} + \epsilon}. \tag{14}$$

The operations in this update are elementwise. The adaptive scaling changes the update applied to each parameter, while the configured base learning rate remains fixed.

No learning-rate scheduler, warm-up phase, or scheduled decay is implemented in the fitting routine. The batch update consists of clearing gradients, a forward pass, loss computation, `backward()`, and `optimizer.step()`. Gradient clipping is not applied. The run limit is 50 epochs, subject to early stopping after five consecutive epochs without a strict improvement in validation macro-F1. Early stopping limits the number of epochs; it does not lower the learning rate. Test evaluation uses the checkpoint with the best recorded validation macro-F1.

Sharing Adam and the same base learning rate provides a common optimization setup for the architecture comparison. It does not establish that this setting is individually optimal for every model. Differences in parameter count, input representation, and architectural regularization must also be considered when interpreting the results.

6.3 Regularization

With weight decay set to zero, the optimizer applies no weight-decay penalty. The models instead differ in their use of dropout and normalization, as summarized in Table 3. The normalization column refers to layers inside the model, rather than the common pixel normalization applied to every input image.

Table 3: Dropout and internal normalization in the reported model configurations.

Model	Dropout	Normalization inside the model
Linear	None	None
MLP	$p = 0.2$ after each hidden ReLU	None
CNN	$p = 0.25$ before the final linear layer	BatchNorm after each convolution
LSTM	None in the saved configurations	None
Transformer	$p = 0.1$ after token preparation and within encoder blocks	LayerNorm in encoder blocks and before the classifier

Dropout randomly removes activations during training and is disabled in evaluation mode. BatchNorm uses batch statistics during training and its accumulated running statistics during evaluation. LayerNorm uses each token's feature statistics in both modes. Normalization is part of the model computation and should not be treated as an explicit weight penalty.

Early stopping and validation-based checkpoint selection are shared across the architectures. Data augmentation is disabled in the main configuration, as described in Section 3. No ablation in the main comparison isolates the effect of dropout or normalization. Consequently, differences between architectures cannot be attributed to architectural family alone while ignoring their different regularization settings.

7 Training and Evaluation Protocol

The protocol below describes the seven recorded runs: the five main architecture configurations, with rows used for the main LSTM, plus its column and patch variants. We checked the configuration snapshots, epoch histories, training metadata, and test outputs against one another. These checks are recorded in `results/a1/tables/a1_protocol_verification.json`; they audit existing artifacts and do not constitute new training or test experiments.

7.1 Training Configuration

Every run uses the fixed Stratified Split and preprocessing described in Section 3. The shared optimization settings are summarized in Table 4; architectural settings and model-specific regularization appear in Sections 5 and 6.

Table 4: Shared training protocol verified from all seven saved configurations.

Setting	Recorded value
Dataset partitions	54,000 train; 6,000 validation; 10,000 official test
Random seed	42
Batch size / workers	128 / 2
Loss / optimizer	Cross-entropy / Adam
Base learning rate	10^{-3}, fixed; no scheduler
Weight decay / augmentation	0 / none
Maximum epochs	50
Early stopping	5 consecutive epochs without improved validation macro-F1
Checkpoint selection	Highest validation macro-F1; strict improvement
Computation	CUDA; standard floating-point training without mixed precision

The common epoch limit provides a shared upper bound rather than an equal number of updates: early stopping can end different models at different epochs. The implementation does not use automatic mixed precision or gradient accumulation; each training batch produces one optimizer step.

7.2 Hardware and Software Environment

All training metadata identify CUDA execution on an NVIDIA GeForce RTX 4060 Laptop GPU. The run-level records consistently report Python 3.10.21, PyTorch `2.14.1+cu126`, and CUDA

12.6. Test outputs also record `device: cuda`. These are the environments of the reported experiments, rather than the local environment used later to draft and compile this report.

The repository README describes the workstation as an AMD Ryzen 7 7735HS laptop with 15.2 GB RAM, an 8 GB RTX 4060 Laptop GPU, and Windows 11. The automatic run records identify the GPU and an AMD64 processor string, but do not independently capture the RAM, GPU-memory capacity, or operating-system version. We therefore distinguish these README-supplied workstation details from the automatically logged software and device fields.

The dependency snapshot is not an exact match to the run metadata: `requirements.lock.txt` lists PyTorch 2.14.0, whereas the recorded runs report `2.14.1+cu126`. Other dependency versions listed in that snapshot should not be assumed to have been captured independently for each run. Reproduction should preserve the recorded CUDA/PyTorch build and reconcile this manifest difference rather than silently treating the snapshot as a complete environment lock.

7.3 Hyperparameter Selection

The recorded comparison uses shared training defaults rather than a documented model-by-model hyperparameter search. Layer widths, dropout rates, and sequence representations are specified in each run's saved configuration. No search history or complete tuning budget is included in the experiment artifacts, so the report does not claim that these settings maximize each architecture's achievable performance.

Validation macro-F1 determines checkpoint selection and early stopping. The test set is evaluated by a separate entry point after loading the selected checkpoint; test scores are not used by the fitting routine. The artifacts document the final settings and outcomes, but do not establish the full history of any manual decisions preceding those runs.

7.4 Checkpoint Selection

After each epoch, the trainer evaluates the complete validation set and compares macro-F1 with the best value seen so far. A strictly larger value overwrites `best.pt` and resets the patience counter. An equal or lower value increments the counter. Training ends after five consecutive non-improving epochs or at the 50-epoch limit, and test evaluation restores the best checkpoint rather than the final epoch's weights.

We confirmed that each saved `best_epoch` and `best_value` matches the corresponding maximum in `history.csv`, and that all seven recorded runs stop after five subsequent non-improving epochs. Stored epoch indices start at zero; an index of 7 therefore denotes the eighth training epoch. Epoch indices quoted from these logs retain this convention.

7.5 Evaluation Metrics

Accuracy and macro-F1 measure predictive quality on the 10,000-image official test set. Parameter count, total training time, and forward-pass inference time describe different aspects of computational cost. A method is assessed using these measures together rather than declaring the largest accuracy alone to be sufficient evidence of superiority.

7.5.1 Accuracy

For N test images, let $\widehat{y}_i$ be the predicted label and y_i the ground truth. Accuracy is

$$\text{Accuracy} = \frac{1}{N}\sum_{i=1}^{N}\mathbb{I}[\widehat{y}_i = y_i]\,. \tag{15}$$

The indicator $\mathbb{I}$ equals one for a correct prediction and zero otherwise. The implementation obtains each prediction by taking the argmax of the ten logits. Accuracy is reported as a fraction or percentage, with the unit indicated in each result table.

7.5.2 Macro-F1

For class k, define TP_k, FP_k, and FN_k as its true-positive, false-positive, and false-negative counts. The classwise F1 and macro average are

$$F1_k = \frac{2TP_k}{2TP_k + FP_k + FN_k}, \qquad \text{Macro-F1} = \frac{1}{10}\sum_{k=0}^{9} F1_k. \tag{16}$$

Scikit-learn computes this quantity with `average="macro"`, giving every class equal weight [8]. It incorporates both precision and recall. Equal class frequencies do not make it interchangeable with accuracy because the distribution of false-positive and false-negative errors can differ across classes.

The saved confusion matrices contain 1,000 test examples per true class. We recomputed accuracy and macro-F1 from these matrices and confirmed agreement with all seven stored metric files. Confusion matrices and prediction examples provide the class-level context for these aggregate scores.

7.5.3 Parameter Count

The training script sums `numel()` over model parameters. All parameters in the configured models are trainable, so this is also their trainable parameter count. It includes weights and biases and excludes buffers such as BatchNorm running statistics. It measures model size in scalar parameters and is not a direct measure of runtime or peak memory.

7.5.4 Training Time

Training time is measured in seconds around the call to `fit`. It includes the training and validation loops, checkpoint writes, and history-file output within that routine. Dataset preparation, model construction, and the separate final test and plotting steps fall outside this interval. The elapsed time is saved as `train_time` in `metrics.json`.

Because early stopping changes the number of completed epochs, this value is the cost of the recorded run rather than a measurement at a fixed update budget. The number of executed epochs is reported alongside runtime when interpreting efficiency.

7.5.5 Inference Time

The timing routine in `src/evaluate.py` measures forward passes over the full test set with batch size 128. It first loads and normalizes the data and transfers the tensors to the target device.

The current implementation performs three warm-up calls on the first batch, then times five full test-set passes in evaluation mode with gradients disabled. CUDA synchronization brackets each timed pass so the interval includes completion of the device work.

The reported duration is the median of the five passes. The saved outputs confirm a batch size of 128 and five repetitions for every run. Data loading, normalization, host-to-device copies, argmax, metric calculation, and plot generation are excluded from this timing. The stored per-image quantity is

$$t_{\text{image}}\ [\text{ms}] = \frac{1000\ \text{median}(t_1, \ldots, t_5)}{10{,}000}. \tag{17}$$

This is an amortized time per image under batched inference, not a separately measured single-image request latency. We checked that the stored per-image times match the total-duration conversion above.

7.6 Fair Comparison Protocol

The main comparison holds the saved split, seed, preprocessing, batch size, optimizer settings, epoch limit, checkpoint criterion, test set, and metric definitions fixed. Recorded training and test device fields also agree across runs. Architectural capacity, input representation, and regularization vary as documented in Section 5; parameters and computation budgets are not matched.

Each configuration has only one recorded random seed, 42. Consequently, the tables describe individual runs and do not provide seed-to-seed standard deviations, confidence intervals, or evidence of statistical significance. Repeating inference timing five times does not provide independent training seeds.

The five main runs record commit `c1747a1`. The additional LSTM column and patch runs record `c1747a1-dirty`, indicating uncommitted source or configuration changes during training. Their saved configurations and metrics remain available, but the commit identifier alone does not recover those working-tree changes. This limits traceability of these additional comparisons. The training commit also does not independently identify a later evaluation-code revision; the timing description above follows the stored timing metadata and the available evaluation implementation.

7.7 Experimental Hypotheses and Controlled Factors

The following questions organize the analysis of the existing runs. They are descriptive comparison criteria, with the common controls stated above; no multi-seed significance test or unrecorded acceptance threshold is assumed.

1. **Nonlinear capacity: MLP versus linear.** The hypothesis is that hidden nonlinear layers improve predictive quality. The changed factors include hidden capacity and dropout. Compare test accuracy and macro-F1, then quantify the accompanying changes in parameters and runtime. An improvement supports a benefit of the configured MLP as a whole, rather than isolating nonlinearity alone.

2. **Spatial processing: CNN versus MLP.** The hypothesis is that local feature extraction and shared filters provide a useful spatial bias. The changed factors include connectivity, capacity, pooling, and normalization. Assess predictive metrics jointly with parameter count and runtime, and use class confusions to locate differences. This is an architecture comparison, not a single-factor ablation.

3. **Scan orientation: LSTM rows versus columns.** Change only the sequence orientation in the saved model settings while keeping hidden size, layer count, and parameter count fixed. Compare macro-F1, accuracy, and runtime to examine sensitivity to scan order. Interpret the result alongside the column run's incomplete commit traceability.

4. **Sequence granularity: LSTM rows versus patches.** The hypothesis is that grouping pixels into patches changes how the recurrent model processes spatial information. Both sequence length and input width change, increasing the parameter count. Compare predictive quality and runtime while reporting this capacity difference; the experiment does not isolate sequence length alone.

5. **Global token interaction: Transformer versus CNN.** Compare the patch encoder with the spatial CNN to examine the trade-off between global self-attention and convolutional processing. Report accuracy and macro-F1 alongside size and timing. Architectural family, tokenization, depth, and regularization all vary, so an observed difference is attributed to the recorded configurations rather than attention alone.

Part III

Implementation Results

8 Training Behaviour

This section examines the saved epoch histories of the five main models and the two additional LSTM variants. All results use seed 42 and the same Stratified Split. The figures replot the recorded values without smoothing; they do not represent new training experiments. Epoch indices start at zero throughout, so checkpoint index 7 is the eighth completed epoch.

8.1 Training and Validation Curves

Figure 9 shows sample-weighted training and validation cross-entropy for each main model. Shared axes make both the loss scale and the different stopping points visible. Each dotted line marks the epoch selected by validation macro-F1, which need not minimize validation loss.

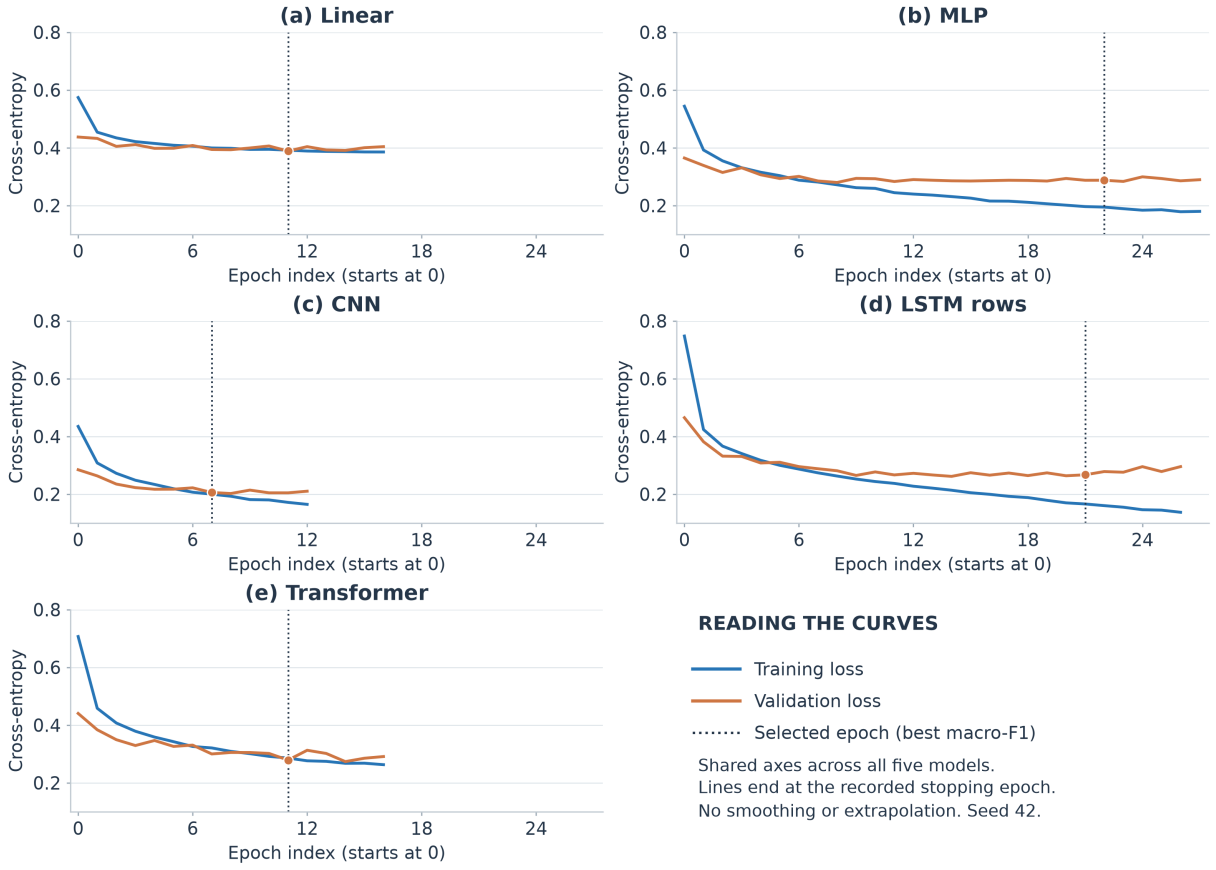


Figure 9: Training and validation loss for the five main configurations. The main LSTM uses row sequences. Curves end at the recorded stopping epoch; a dotted line and validation-loss marker locate the selected checkpoint. Source: each run's saved `history.csv`.

All five models reduce training loss between the first and final recorded epochs. The CNN reaches a validation loss of approximately 0.20 early in its run, whereas the linear model remains near 0.39–0.40. For the MLP and row LSTM, training loss continues to fall after the lowest validation loss has been reached, opening a larger separation between the curves.

Figure 10 displays the actual checkpoint-selection metric. The CNN attains the highest peak validation macro-F1, 92.68%, at index 7. The MLP and row LSTM reach their selected peaks later, at indices 22 and 21. These are validation observations; the held-out test comparison is reported separately in the next section.

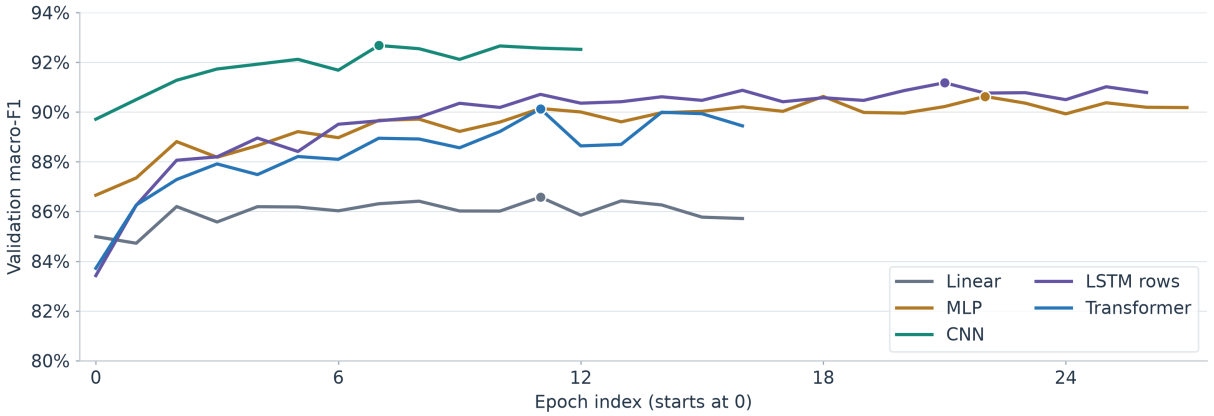


Figure 10: Validation macro-F1 of the five main configurations on 6,000 validation images. Filled markers indicate the selected checkpoints. Each line stops at its own final recorded epoch; no values are extended to the common 50-epoch limit.

8.2 Convergence and Checkpoint Selection

Table 5 separates the number of completed epochs from the zero-based checkpoint index. All seven runs stopped after five consecutive epochs without a strictly higher validation macro-F1; none exhausted the 50-epoch limit. This establishes the recorded stopping condition, rather than convergence to an optimum.

Table 5: Observed stopping and checkpoint selection. Epoch columns contain zero-based indices; completed epochs are counts. Peak F1 is validation macro-F1, expressed as a percentage.

Configuration	Completed epochs	Selected epoch	Min. val. loss epoch	Peak F1 (%)
Linear	17	11	11	86.58
MLP	28	22	8	90.63
CNN	13	7	8	92.68
LSTM rows	27	21	14	91.18
Transformer	17	11	14	90.13
LSTM columns	26	20	15	90.42
LSTM patches	26	20	11	90.17

The MLP illustrates why the two epoch criteria differ: its minimum validation loss is 0.2799 at index 8, while its best macro-F1 occurs at index 22 with validation loss 0.2876. Cross-entropy depends on the probabilities assigned to the labels, whereas macro-F1 depends on the predicted classes. Improvements in the latter therefore need not coincide with reductions in the former. The row LSTM and Transformer show the same mismatch between the two selected indices, although the ordering of those indices differs.

The final epoch is also not the selected checkpoint. For example, CNN validation macro-F1 falls from its peak of 92.68% to 92.52% at the final index 12. The saved best checkpoint is restored for test evaluation.

8.3 Overfitting and Underfitting Analysis

Interpreting the loss gap. Training loss is averaged over batches while the parameters are changing, with dropout and normalization layers in training mode. Validation loss is evaluated after the epoch with fixed parameters, dropout disabled, and BatchNorm using its running statistics where present. The difference between these losses is therefore not a controlled generalization-gap measurement. In particular, a validation loss below training loss, as seen at some early epochs and at the selected Transformer checkpoint, is not by itself evidence of data leakage or a faulty split.

Main model behaviour. At the selected MLP checkpoint, training and validation losses are 0.1946 and 0.2876. By the final epoch, training loss falls to 0.1797 while validation loss rises to 0.2894 and macro-F1 decreases from 90.63% to 90.18%. For the row LSTM, the corresponding selected losses are 0.1661 and 0.2672; final losses become 0.1373 and 0.2955. These trends are consistent with increasing overfitting late in the recorded runs. The CNN shows a smaller selected loss separation, 0.1999 versus 0.2059, followed by a larger separation at the stopping epoch. Early stopping retains the higher-F1 checkpoint in each case.

The linear model retains a training loss of 0.3917 at its selected checkpoint and reaches a lower peak validation F1 than the nonlinear models. Its relatively high training loss and closely tracking validation loss are consistent with limited capacity under the recorded setup. They do not establish that a different optimizer, learning rate, or training budget could not improve it. Likewise, the Transformer curves alone do not identify an architectural cause for its validation result; no controlled regularization or optimization ablation was recorded.

LSTM input representations. Figure 11 compares all three sequence representations. The patch variant reaches the lowest final training loss, 0.1067, but the highest final validation loss, 0.3516. Its loss separation increases from 0.1765 at the selected checkpoint to 0.2450 at the final epoch. Rows and columns end with smaller separations of 0.1582 and 0.1356, respectively. This makes the patch configuration's late overfitting particularly visible, despite its better fit to the training batches.

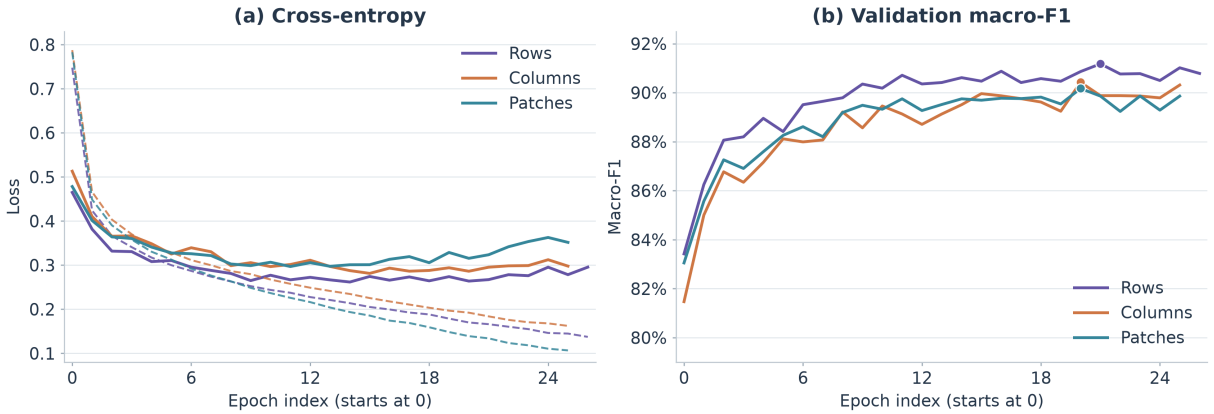


Figure 11: LSTM representation comparison. Left: validation loss (solid) and training loss (dashed). Right: validation macro-F1, with selected checkpoints marked. All three histories use seed 42. Patch inputs change both the sequence representation and parameter count, so the comparison is not a capacity-matched ablation.

The row configuration has the highest peak validation macro-F1 among the three, at 91.18%, compared with 90.42% for columns and 90.17% for patches. These are single-run observations.

The seed and commit-traceability limits in Section 7 also apply here; the figures do not establish a statistically reliable ranking across random seeds.

8.4 Implementation Issues and Fixes

The development record in `AI_USAGE.md`, Entry 2, documents several issues encountered while building the training and evaluation pipeline:

1. **Writing history after training.** A missing `pandas` import caused `fit()` to fail after training. The current trainer imports `pandas` and writes the epoch records to `history.csv`, making the curves and checkpoint checks above recoverable from saved artifacts.
2. **Plotting and console output.** Missing plotting imports and a Windows cp1252 console error when printing Vietnamese were documented and fixed during development. The current plotting module contains the required imports. These are artifact-generation and output-encoding issues; the record does not provide a controlled measurement of any effect on predictive accuracy.
3. **Inference timing on short loaders.** The development record reports a division-by-zero error in the earlier timing routine. The current evaluator warms up separately, times repeated complete passes, and derives per-image duration from the image count, as described in Section 7. This avoids estimating the timed workload by subtracting warm-up batches from the loader length.

These are documented development fixes, not failures reproduced during report preparation. For this section, verification consists of checking saved histories against run metadata and inspecting the current implementation. No new training or before-and-after predictive experiment was performed.

9 Quantitative Results

The results below use the 10,000-image official test set and the checkpoints selected by validation macro-F1. We recomputed accuracy, macro-F1, and classwise metrics from the saved test confusion matrices and checked agreement with the recorded comparison table. This is an analysis of existing evaluation artifacts; no new predictions were generated.

9.1 Main Model Comparison

Table 6 reports predictive performance and computational cost together. The first five rows form the main architecture comparison; the final two provide the additional LSTM representations. Accuracy and macro-F1 are percentages, and all rows use seed 42.

Table 6: Test results and recorded computational cost. Bold values identify the highest accuracy and macro-F1 among these runs. Training time includes validation and checkpointing; inference is amortized forward-pass time per image at batch size 128.

Configuration	Acc. (%)	Macro-F1 (%)	Parameters	Epochs	Train (s)	Inference (ms/image)
Linear	84.03	84.08	7,850	17	508.2	0.00052
MLP	89.38	89.38	235,146	28	784.8	0.00236
CNN	**91.82**	**91.76**	50,378	13	367.5	0.00349
LSTM rows	89.92	89.89	82,186	27	756.6	0.00458
Transformer	88.46	88.44	540,170	17	502.3	0.01334
LSTM columns†	88.61	88.66	82,186	26	730.2	0.00472
LSTM patches†	88.33	88.38	92,938	26	728.1	0.00374

† The column and patch LSTM runs record `c1747a1-dirty`; their exact working-tree changes are not recoverable from the commit identifier alone. Timing follows the protocol and limits in Section 7.

The CNN has the highest recorded test accuracy, 91.82%, and macro-F1, 91.76%. It correctly classifies 9,182 images, leaving 818 errors. Its accuracy exceeds the linear baseline by 7.79 percentage points, the MLP by 2.44 points, and the row LSTM by 1.90 points. These differences describe the saved runs, without a claim of statistical significance across training seeds.

The linear model uses the fewest parameters and has the lowest recorded inference time. The CNN has fewer parameters than the MLP and the shortest total training run, but completes only 13 epochs compared with the MLP's 28. Total training duration therefore cannot be read as an equal-budget speed comparison. The Transformer has the most parameters and the highest forward-pass time among these configurations; its test accuracy is below the MLP, CNN, and row LSTM results.

Rows give the highest test scores among the recurrent representations. Rows and columns share the same parameter count, whereas patches increase it. Further interpretation of these trade-offs is reserved for the model discussion.

9.2 Classwise Performance

Table 7 gives test F1 for each of the ten classes, computed from the saved matrices using the definitions in Section 7. Each true class has exactly 1,000 test images. The final macro-F1 is the arithmetic mean of the ten unrounded class scores; rounding the displayed cells can introduce a small difference in a manually recomputed average.

Table 7: Classwise test F1 (%) for the five main configurations. Bold marks the largest unrounded value in each row. Every class has support 1,000.

Class	Linear	MLP	CNN	LSTM rows	Transformer
T-shirt/top	79.73	84.86	**87.71**	83.84	83.97
Trouser	96.68	97.99	**98.95**	97.92	97.92
Pullover	73.13	80.76	**87.70**	84.05	80.24
Dress	85.12	89.63	**90.90**	89.45	88.67
Coat	74.65	81.62	**86.99**	83.82	79.54
Sandal	92.83	96.88	**98.35**	97.96	96.95
Shirt	58.94	73.33	**75.54**	71.47	69.59
Sneaker	91.88	94.89	**96.45**	95.91	94.65
Bag	93.66	97.83	**98.01**	98.00	96.73
Ankle boot	94.14	96.02	**97.02**	96.50	96.11

Shirt is the lowest-F1 class for every main model: 58.94% for linear, 73.33% for the MLP, 75.54% for the CNN, 71.47% for the row LSTM, and 69.59% for the Transformer. Even the best aggregate result therefore leaves a substantial weakness on this class. For the CNN, Shirt precision is 80.07% and recall is 71.50%, so 285 of the 1,000 true Shirt images are missed.

The CNN has the largest classwise F1 in all ten rows of this recorded comparison. Its Trouser, Sandal, Bag, and Ankle boot F1 scores exceed 97%, while T-shirt/top, Pullover, Coat, and Shirt remain below 90%. The aggregate score consequently hides considerable class-level variation. Complete precision, recall, F1, and support values for all seven configurations are exported to `results/a1/tables/a1_classwise_metrics.csv`.

9.3 Confusion Matrices

Figure 12 compares the five main models using a common color scale. Rows are true classes and columns are predictions. Each row is divided by its 1,000 true examples, allowing the same cells to be compared directly across configurations.

The CNN's largest directed confusion is Shirt → T-shirt/top: 112 cases, or 11.2% of all true Shirts. Other frequent Shirt errors are Coat (82 cases) and Pullover (61 cases). Together, these three destinations account for 255 of its 285 missed Shirts. The reverse T-shirt/top → Shirt direction contains 59 cases, so the confusion is not symmetric. Pullover → Coat adds 54 cases. These counts locate the remaining errors; visual explanations require the image-level analysis in the following sections.

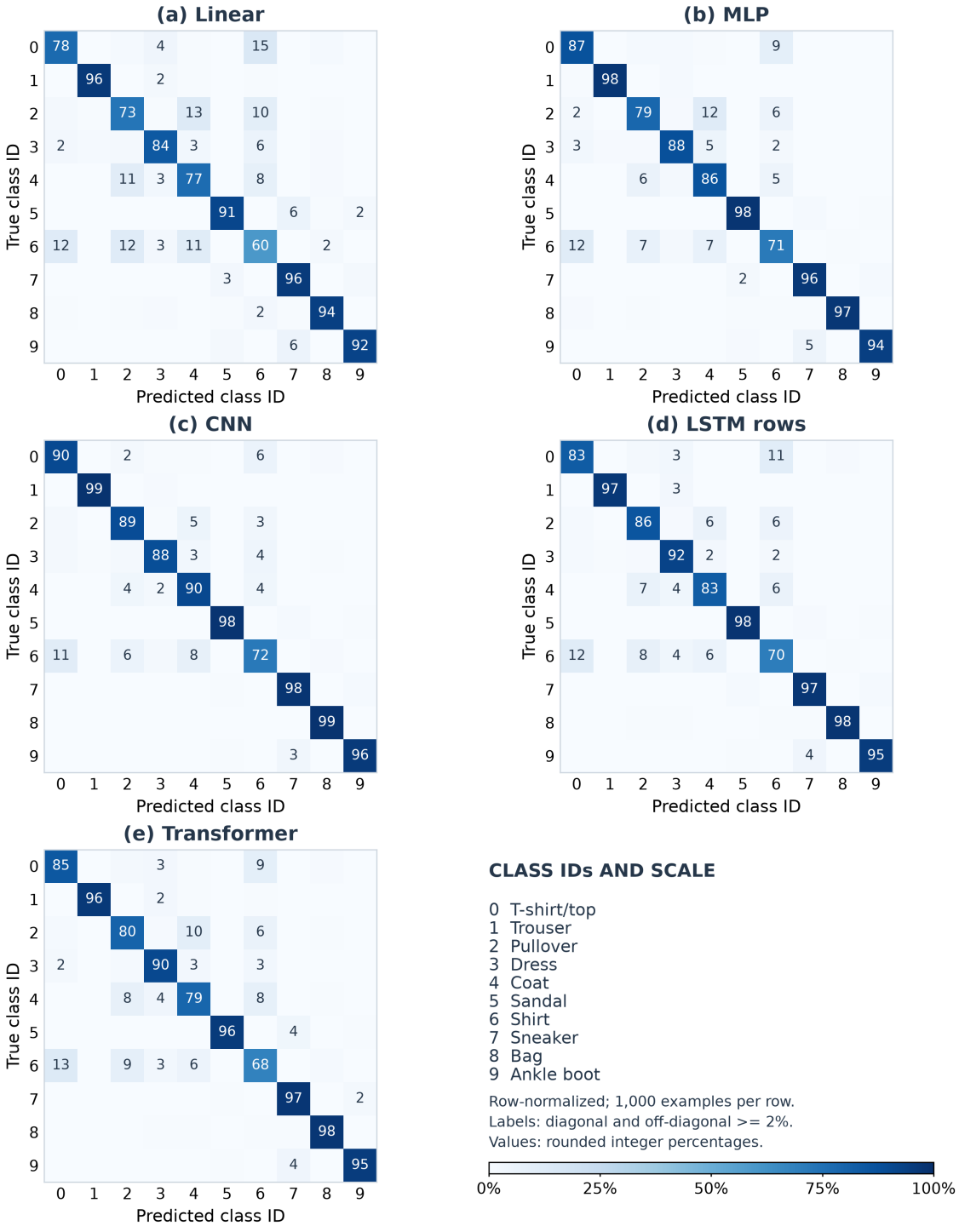


Figure 12: Row-normalized test confusion matrices for the five main configurations. Class IDs follow the key. Cell labels show percentages rounded to integers, restricted to diagonal cells and off-diagonal rates of at least 2%; unlabeled cells may still contain errors. Colors use the unrounded rates on the shared 0–100% scale.

9.4 Variation Across Random Seeds

Only one training seed, 42, is available for each configuration. The `n_seeds` field in the saved summary is one, and sample standard deviations are absent. We therefore report individual-run values without a mean ± standard deviation, confidence interval, or error bars across seeds. The two extra LSTM representations are distinct configurations, not repeated seeds of the row model. Likewise, five timing repetitions measure repeated forward passes and do not estimate variation caused by training initialization. Multi-seed stability remains unmeasured.

10 Qualitative Results

We examine CNN run `a1_cnn_s42_1027-0157`, which has the highest recorded test scores (Section 9). Cases are selected from its saved grids of the first 16 correct and first 16 incorrect predictions in test-set order. They illustrate individual outcomes rather than population frequencies or robustness.

All 32 source panels were matched to original test images and their true labels checked. Predictions come from saved annotations; no checkpoint was rerun. Figures show original 28×28 pixels with fixed grayscale and nearest-neighbor enlargement. Test IDs are zero-based; panel provenance is recorded in `results/a1/tables/a1_qualitative_examples.json`.

10.1 Correct Predictions

Figure 13 shows six correct predictions. The trouser image has two distinct legs, the dress has a long sleeveless outline, and the footwear shows different silhouettes. The correctly classified Shirt illustrates success on an individual example from the CNN's lowest-F1 class.

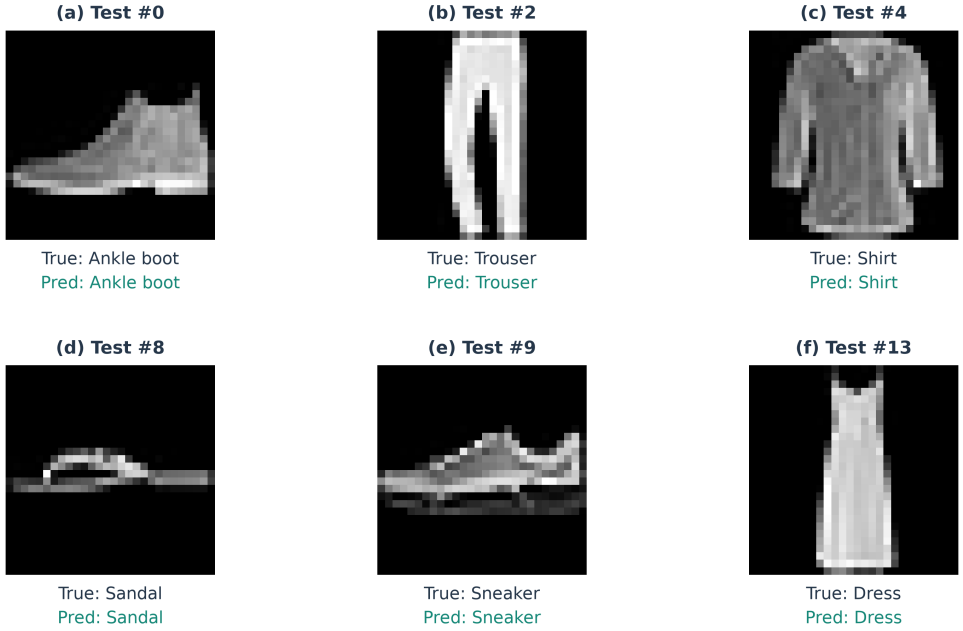


Figure 13: Six correct CNN predictions selected from the saved correct-example grid. Each panel gives its official test index, ground-truth label, and recorded prediction. These examples illustrate individual successes rather than a balanced sample of all ten classes.

10.2 Difficult but Correct Cases

Here, "difficult" describes visually challenging characteristics identified during inspection, not low confidence or a measured difficulty score. Figure 14 shows four further correct cases, without repeating the images in Figure 13.

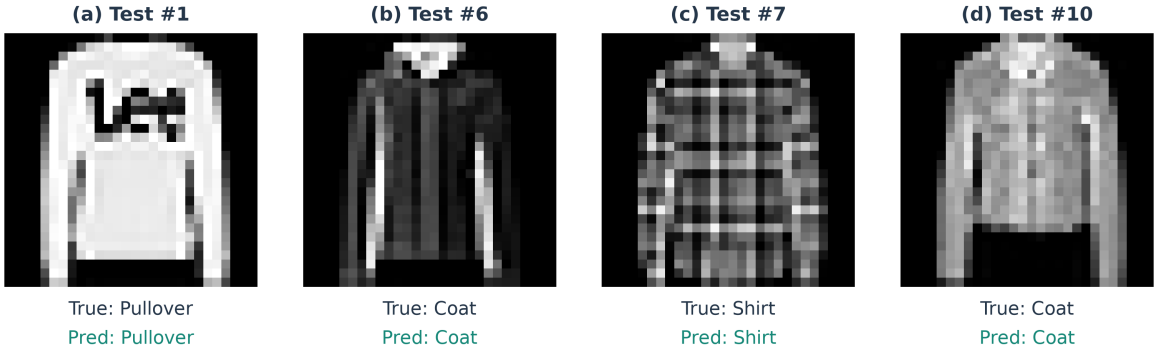


Figure 14: Visually challenging but correct CNN cases: a pullover with a prominent chest graphic (test #1), a dark coat (#6), a checked shirt (#7), and a lighter coat (#10). The cases were chosen by visual inspection, without confidence-based ranking.

The chest graphic in test #1 introduces strong internal contrast, while the checked pattern in test #7 creates many local intensity changes. Both are classified correctly despite their different surface patterns. The two Coat images, tests #6 and #10, vary visibly in brightness and contrast while retaining the same correct predicted class. These observations demonstrate success on these particular images; they do not establish invariance to arbitrary texture or lighting changes because no controlled perturbation experiment was performed.

The correctly classified checked Shirt is also informative when compared with test #248 in Figure 15: that checked Pullover is predicted as Shirt. A similar visible pattern can therefore accompany both a correct and an incorrect result. This comparison motivates examining shape and texture together, but does not identify a causal feature without additional model analysis.

10.3 Incorrect Predictions

Figure 15 presents eight errors selected from the saved incorrect-example grid. Their labels remain the official ground truth and the recorded CNN prediction, even where an image is visually ambiguous. No relabeling is inferred from appearance alone.

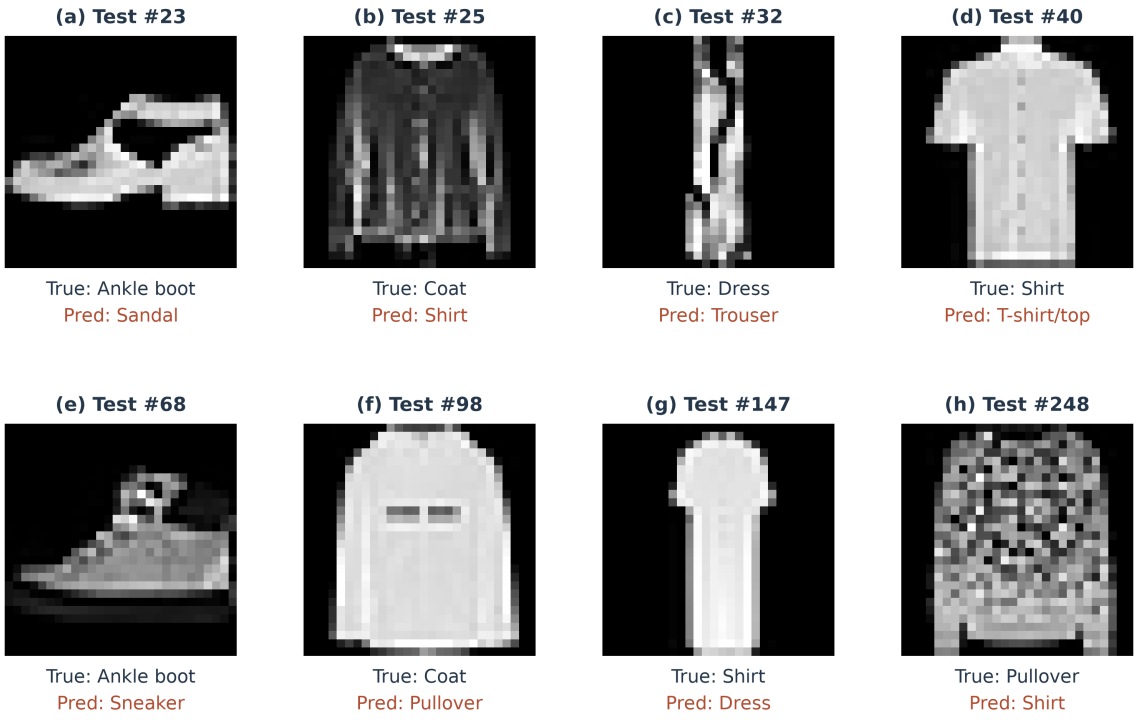


Figure 15: Eight incorrect CNN predictions, selected to illustrate upper-body garment overlap, narrow or elongated views, and footwear confusion. The labels and test IDs identify individual observed errors; their proportions in this selected figure do not estimate error frequencies.

Upper-body garment overlap. The Coat in test #25 is predicted as Shirt, the Coat in #98 as Pullover, and the Pullover in #248 as Shirt. All have a torso-and-sleeve outline, while small construction details are difficult to resolve at 28×28. Test #40, officially Shirt but predicted as T-shirt/top, has a short-sleeved appearance. These examples are consistent with the garment confusions in the test matrices, although they do not prove that low resolution alone caused the errors.

View and silhouette ambiguity. The Dress in test #32 appears as a narrow view and is predicted as Trouser. The Shirt in #147 has an elongated light outline and is predicted as Dress. These images illustrate cases where the visible silhouette provides incomplete or overlapping category cues. They do not establish that either official label is incorrect.

Footwear confusion and selection limits. Tests #23 and #68 are Ankle boots predicted as Sandal and Sneaker, respectively. Their visible openings, ankle regions, and sole shapes invite comparison with the correctly classified footwear in Figure 13. However, the early-error grid is not a frequency summary: the complete CNN matrix contains 112 Shirt-to-T-shirt/top errors and 34 Ankle-boot-to-Sneaker errors. Conclusions about prevalence therefore come from Section 9, rather than the number of shoe images selected here.

The saved figures provide no per-image probability or logit values, and this analysis computes no saliency or attribution maps. Explanations above are visual hypotheses grounded in identified examples, not claims about confidence or the model's internal attention. The later error-analysis section can organize these observations alongside the full-set confusion counts.

11 Model Comparison and Discussion

The comparison combines three types of evidence: whole-test-set metrics and confusion counts, the saved learning curves, and identified examples from the result images. Architectural explanations are checked against the implemented operations. A measured difference establishes what happened in these seed-42 runs; agreement with an architectural mechanism makes an explanation plausible, but does not isolate its causal contribution. Shared preprocessing and optimizer settings control part of the experiment, while capacity, normalization, dropout, representation, and completed epochs still differ. The seed and commit-traceability limits in Section 7 apply throughout.

11.1 Performance Differences

Nonlinear models improve the recorded baseline. The MLP improves accuracy over the linear classifier by 5.35 percentage points and macro-F1 by 5.30 points, reducing the error count from 1,597 to 1,062. This is a 33.5% relative reduction in errors. The CNN reduces the linear baseline's errors further, to 818: 779 fewer errors, or a 48.8% relative reduction. These are net count differences on the same 10,000 test images, not counts of individually identified corrections; an aggregate matrix does not reveal which images one model corrects while another loses.

The CNN's advantage is concentrated, not uniform. For descriptive analysis, define the five-class group G as T-shirt/top, Pullover, Dress, Coat, and Shirt. This grouping is chosen from the observed garment-confusion pattern, rather than declared as a separate evaluation split. It contains 5,000 test images, exactly half the test set. Of the CNN's 818 errors, 678 (82.9%) are confusions between two different classes within G. Counting all errors whose true class lies in G gives 711: an error rate of 14.22%, versus 2.14% on the other 5,000 images. Equal class support therefore does not imply equal difficulty, and class-frequency imbalance cannot explain this concentration.

Figure 16 decomposes the CNN's net error advantage by true class. Against the MLP, Pullover contributes 99 fewer misses and Coat 39, while Shirt contributes only three. In total, the true classes in G account for 178 of the 244 net fewer errors (73.0%). The main aggregate improvement therefore comes from reducing mistakes on several overlapping garment classes, rather than eliminating the Shirt problem.

Higher macro-F1 does not mean higher recall everywhere. For Dress, the row LSTM has recall 92.40%, compared with 88.40% for the CNN: 76 versus 116 missed Dresses. However, the row LSTM also produces 142 false-positive Dress predictions, compared with 61 for the CNN. Their Dress precisions are consequently 86.68% and 93.54%, and F1 favors the CNN, 90.90% versus 89.45%. The CNN leads the classwise F1 table while the row LSTM retrieves more true Dresses. This is a concrete reason to examine precision and recall before declaring a model better for every class-level objective.

The Shirt comparison also qualifies the aggregate gain. CNN recall is 71.50%, only 0.30 points above the MLP's 71.20%, whereas precision rises from 75.58% to 80.07%. Its higher Shirt F1 accompanies fewer false positives, while 285 true Shirts are still missed. The saved confusion matrices support improvement, but not a claim that this difficult category has been solved.

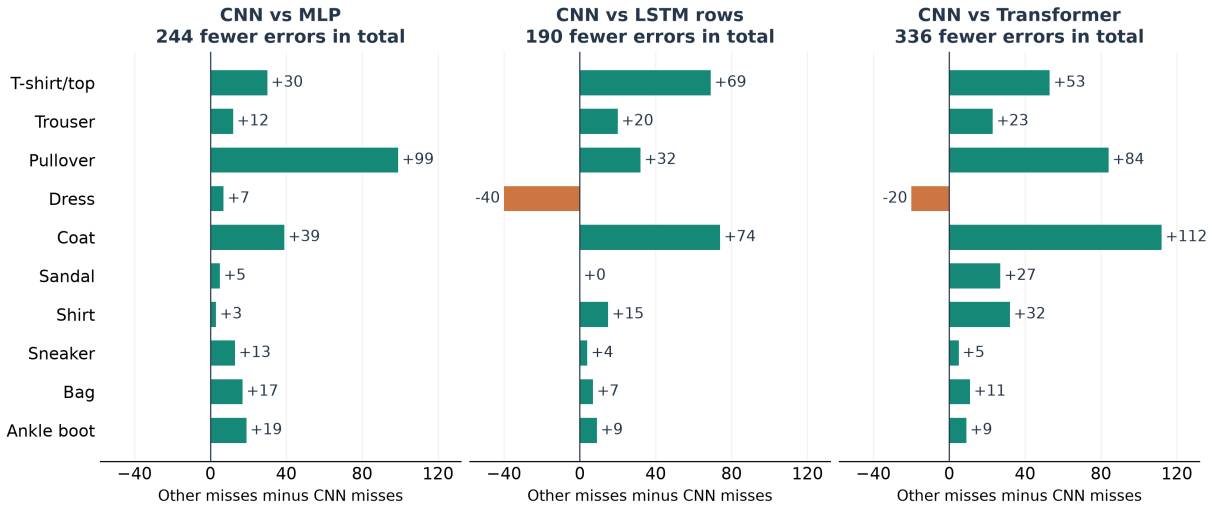


Figure 16: Net difference in missed test images per true class: other-model misses minus CNN misses. Positive bars favor the CNN; negative bars favor the comparator. Each class has support 1,000. The bars sum to the overall difference in error counts, but do not identify paired corrections on individual images.

11.2 Accuracy, Speed, and Parameter Trade-offs

The useful choices depend on the cost being constrained. Figure 17 plots all seven configurations. A point is *non-dominated* if no recorded configuration has both at least as high a macro-F1 and no greater cost, with a strict improvement in one. For parameters versus macro-F1, only Linear and CNN are non-dominated. For batched inference time versus macro-F1, MLP also remains on the frontier: it is faster than the CNN, although less accurate. The CNN is better than each recorded LSTM and the Transformer on both macro-F1 and forward-pass time. The frontiers use saved point estimates; between-seed and timing-dispersion estimates are unavailable. They concern the measured configurations and two stated objectives, rather than a universal architecture ranking.

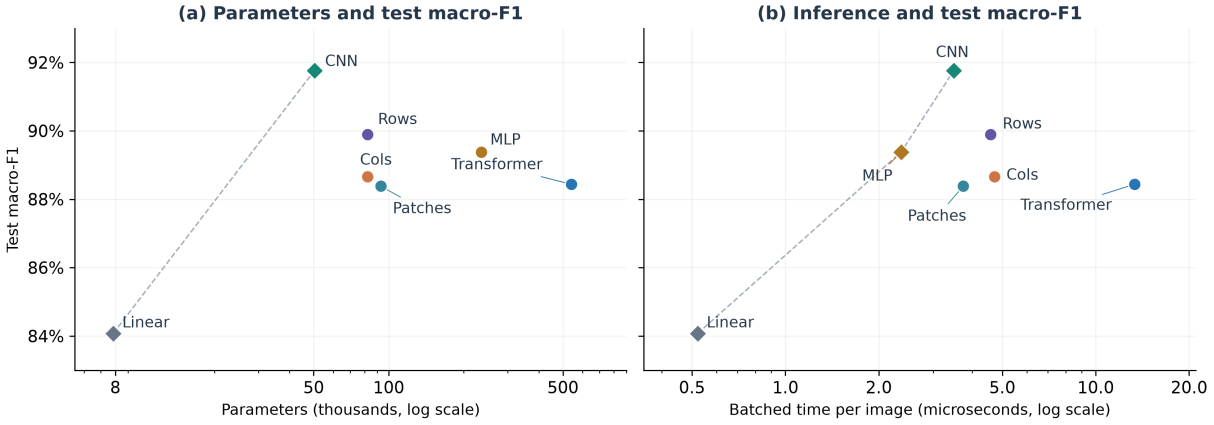


Figure 17: Quality and cost for the seven saved runs. Diamonds mark non-dominated configurations for each pair of objectives; dashed segments connect those observations as visual guides. Rows, Cols, and Patches denote LSTM representations. Time is the saved median batched forward-pass duration divided by 10,000 images, converted to microseconds; it excludes loading and transfers. Logarithmic horizontal axes spread the different cost scales.

Parameter sharing reduces model size, not necessarily arithmetic. The MLP has 4.67 times as many parameters as the CNN, but the CNN's recorded per-image forward time is 1.48 times larger. A filter's parameters are reused at many image locations, so fewer stored weights need not imply fewer operations. Table 8 gives a reproducible structural estimate: the CNN performs approximately 3.87 million multiply-accumulates (MACs) per image, compared with 0.235 million for the MLP, a factor of 16.49. This explains how a compact convolutional model can require more arithmetic than a larger dense model. It is compatible with the timing direction, but does not predict the time ratio because parallel execution, kernels, and memory traffic are not profiled here.

Table 8: Analytical MAC counts per image from configured layer dimensions. One MAC denotes one multiplication accumulated into a sum. Counts include dense/convolutional products, LSTM gate matrix products, and Transformer attention products; they exclude biases, nonlinearities, normalization, pooling, softmax, and data movement. These are operation estimates, not measured hardware FLOPs or latency.

Configuration	MACs per image
Linear	7,840
MLP	234,752
CNN	3,869,824
LSTM rows	2,237,696
Transformer	9,310,464
LSTM columns	2,237,696
LSTM patches	1,451,264

For the Transformer, let $N = 17$ tokens, embedding width $d = 128$, and feed-forward width $m = 256$. The matrix products in one encoder block require

$$M_{\text{block}} = 4Nd^2 + 2N^2d + 2Ndm. \tag{18}$$

The query/key/value and output projections contribute 1,114,112 MACs, the feed-forward network another 1,114,112, and the two attention-mixing products 73,984. Thus, the quadratic token-mixing term is only 3.2% of this block's counted MACs. With such a short sequence, blaming the measured runtime primarily on quadratic attention is not supported by the arithmetic. Four encoder blocks and their dense projections provide a more specific structural account; determining the actual runtime bottleneck would require profiling.

Early stopping also changes the training-time comparison. The CNN's 367.5-second fit is shorter than the MLP's 784.8 seconds, but the runs complete 13 and 28 epochs. Across all seven configurations, total fit time divided by completed epochs ranges from 28.0 to 29.9 seconds. This derived average includes training, validation, loading within the loops, and checkpoint work; it is not an isolated epoch-kernel benchmark. The saved times measure the cost of each architecture together with its stopping trajectory. They do not establish equal-budget training throughput, peak memory, or single-image request latency.

11.3 Effect of Input Representation

Flattening changes the model's assumptions, not the available pixels. Flattening retains all 784 pixel values in a fixed order. The linear model and MLP therefore receive the same image information as the CNN, but their dense layers do not impose local weight sharing. The linear classifier can only form affine logit differences between classes; the MLP can combine

pixels nonlinearly. The higher linear training loss and lower validation/test scores are consistent with a restrictive decision family under this setup. They do not establish the best achievable linear score under every optimizer or tuning budget.

Row versus column scanning is the closest representation control. Both LSTM scans contain 28 steps of width 28, use hidden size 128, and have 82,186 parameters. A row token presents horizontal pixel relations together and recurrence connects successive vertical positions; columns reverse these roles. Rows exceed columns by 1.31 accuracy points and 1.23 macro-F1 points in the recorded runs. This is evidence of sensitivity to scan orientation with matched capacity. It does not prove that vertical progression is intrinsically superior: only seed 42 is available, and the column run records a dirty working tree, limiting exact source reconstruction.

Shorter patch sequences improve one cost while weakening this run's fit to held-out data. The patch LSTM uses 16 steps of width 49. Like the row and column transforms, this is a rearrangement of all 784 pixels, with no downsampling or discarded raw pixels. It changes which pixels are processed together, the number of recurrent updates, and the gate input size. Its parameter count rises by 13.1%, while the counted MACs fall by 35.1% and measured inference time falls by 18.4% relative to rows. The input-to-gate products still require $784 \times 4 \times 128 = 401{,}408$ MACs in both representations; fewer recurrent hidden-state updates account for the arithmetic saving. Test accuracy nevertheless drops by 1.59 points and macro-F1 by 1.51 points. The shorter sequence is therefore a computational trade-off in these results, rather than an unqualified improvement.

The saved curves in Figure 11 strengthen that interpretation: patch LSTM final training loss is 0.1067, below the row model's 0.1373, but final validation loss is higher, 0.3516 versus 0.2955. This pattern is consistent with greater late overfitting, not a simple inability to reduce training loss. Differences in training/evaluation mode still apply, and representation, capacity, and update sequence change together. A patch-size or regularization ablation would be needed to isolate their contributions.

11.4 Effect of Inductive Bias

Linear and MLP: useful baselines with different expressive power. The linear classifier directly scores fixed pixel positions, giving the smallest model and fastest recorded inference. Its 84.03% accuracy shows that these normalized, isolated-item images contain useful class information even for an affine decision rule. The MLP improves the score while preserving the same flattened input. Hidden nonlinear layers can represent interactions between pixels, but the experiment also increases capacity and introduces dropout, so the 5.35-point improvement cannot be attributed to activation functions alone. The MLP's increasing late loss separation in Figure 9 also shows that added flexibility does not remove the need for validation-based stopping.

CNN: a spatial constraint that matches the observed error reductions. Shared local filters reuse the same learned detector across positions; local connectivity and subsampling are established architectural principles for image recognition [9]. Here, two convolution/pooling stages construct a 7×7 feature grid, and the flattened head integrates the entire grid. The model can therefore combine global shape information without self-attention. On small grayscale item images, this structure supplies reusable local feature processing while retaining spatial organization for the final decision. The observed Pullover and Coat improvements are consistent with

this explanation, and the lower parameter count rules out "more parameters" as the explanation of its advantage over the MLP.

However, the run also uses BatchNorm and head dropout, and no capacity-matched dense model or normalization ablation is recorded. Locality alone has not been isolated. Pooling can remove positional detail, and the remaining Shirt confusions show a clear limit of the configured system. Neither the code nor the displayed examples establish full translation invariance, texture robustness, or the exact cues learned by a filter.

LSTM: effective sequence aggregation with an imposed scan. The row model improves on the linear baseline and slightly exceeds the MLP in aggregate scores. Gates and a final hidden representation provide a mechanism for combining information across the image, but the scan orders a two-dimensional object and has sequential state dependencies. Its 128-dimensional final state summarizes the whole sequence. This design differs from the CNN's retained spatial feature grid and the Transformer's token interactions. These differences explain what information-processing constraints are imposed; the available results do not show that the LSTM forgets a particular image region or that recurrence alone causes its lower score than the CNN.

Transformer: greater flexibility is not automatically useful under the shared recipe. The 540,170-parameter model attends globally across 16 patches plus CLS, with learned positions and four encoder blocks. Unlike the CNN, it does not impose sliding local filters or convolutional translation equivariance. The original ViT study discusses weaker image-specific inductive biases and the effect of pretraining scale [10]; its architecture and training recipe differ from our four-block, post-normalized encoder. This provides a plausible reason to examine data efficiency when learning from scratch on 54,000 training images. Its ImageNet-scale findings do not specify a minimum dataset size for this smaller Fashion-MNIST model, nor prove that data quantity caused the observed deficit.

The recorded curves give limited support for diagnosing the Transformer as the most overfit model. At its selected checkpoint, training and validation losses are 0.2858 and 0.2789; the final separation is only 0.0282, compared with 0.1097 for the MLP. These are differently regularized, differently measured losses, not equal-mode generalization gaps, but they do not support explaining the Transformer deficit through the largest visible loss gap. Constant learning rate, no warm-up, no augmentation, and no model-specific tuning are recorded. Optimization, regularization, patch granularity, and inductive bias remain competing explanations. Smaller/overlapping patches or a convolutional stem and separately tuned schedules are testable alternatives, not improvements demonstrated by this report.

What the saved example images actually support. The correct-example grids for Linear, MLP, row LSTM, and Transformer are pixel-identical and show the same first 16 successes. The CNN's grid includes test #12 where those four grids include #16. Consequently, the useful visual comparison is made on common image IDs in Figure 18 and Table 9, rather than treating the success grids as different samples. All displayed predictions below are present explicitly in the corresponding saved correct/incorrect grids; no missing-panel prediction is inferred.

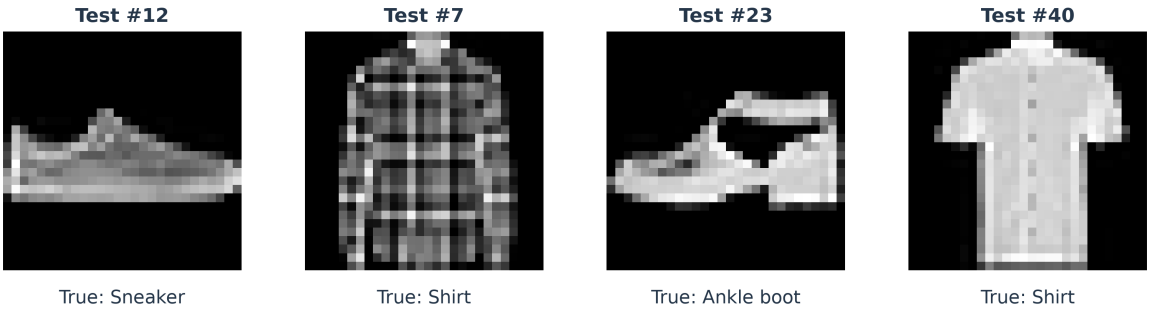


Figure 18: The same four original test images used for the cross-model comparison. The labels are official ground truth; the five sets of recorded predictions appear in Table 9. Images retain their original pixels and are enlarged with nearest-neighbor display.

Table 9: Recorded predictions for the shared examples, recovered from each run's saved example annotations. Bold indicates agreement with ground truth. Cases are deliberately selected illustrations, not an estimate of error overlap across the full test set.

Configuration	Test #12	Test #7	Test #23	Test #40
Linear	Sandal	**Shirt**	Sneaker	T-shirt/top
MLP	Sandal	**Shirt**	Sandal	T-shirt/top
CNN	**Sneaker**	**Shirt**	Sandal	T-shirt/top
LSTM rows	Sandal	**Shirt**	Sandal	T-shirt/top
Transformer	Sandal	**Shirt**	Sneaker	T-shirt/top

Only the CNN recognizes Sneaker #12 correctly; all four alternatives label it Sandal. All five recognize the checked Shirt #7, while all five label Shirt #40 as T-shirt/top. For Ankle boot #23, every model is wrong, but the alternatives divide between Sneaker and Sandal. These cases demonstrate one CNN-specific success and shared failures without assigning their cause to an unobserved filter, attention map, or confidence score. They also show why a good aggregate model still needs class-level and image-level inspection. The selected panels cannot supply full paired error counts or a significance test.

11.5 Answers to Research Questions

1. **How much does nonlinearity improve the baseline?** The configured MLP gains 5.35 accuracy points and 5.30 macro-F1 points over Linear; the CNN gains 7.79 and 7.69 points. These support the benefit of the configured nonlinear models as a whole. They do not isolate nonlinearity from their extra parameters and regularization.

2. **How do spatial and sequence processing relate to quality?** The CNN leads aggregate scores with fewer parameters than the MLP and sequence models. Class counts locate much of its gain in Pullover and Coat, while Dress recall favors the row LSTM. Local spatial processing is a code-consistent explanation, with normalization and optimization still confounded; no architecture dominates every recall objective or every image.

3. **What changes with LSTM representation?** Rows lead columns and patches in this seed-42 comparison. Rows versus columns holds size fixed, while patches shorten the sequence and reduce arithmetic/inference time at the cost of worse test scores and greater late loss separation. None of these reshapes removes raw pixel values, and only the recorded representations have been tested.

4. **What is the quality–cost trade-off?** CNN is the strongest recorded choice for macro-F1 together with parameter economy. Linear remains useful at the smallest size and lowest measured forward time; MLP provides an intermediate speed/quality option. The LSTM and Transformer configurations are dominated by CNN on the plotted quality/time objectives. These statements apply to batch-128 CUDA inference under the protocol, not unmeasured deployment workloads.

Stronger causal conclusions require repeated seeds and matched comparisons: equal-capacity dense/spatial models with normalization and dropout controls; representation changes with explicit capacity controls; and architecture-specific learning-rate/regularization searches with comparable validation budgets. Full per-image predictions would also enable paired error analysis. The current evidence supports a precise ranking of recorded outcomes and several testable mechanisms, without claiming that the architecture families have reached their best attainable performance. Derived comparisons and operation counts are saved in `results/a1/tables/a1_comparison_analysis.json`.

12 Error Analysis

The CNN has the strongest aggregate scores, but still makes 818 errors on 10,000 test images. We analyze these recorded errors using label-pair counts and identified examples. Frequency estimates come from the complete confusion matrix, while visual observations come from selected images; the latter do not estimate the prevalence of a visual cause.

12.1 Error Taxonomy

We use the garment group G defined in Section 11: T-shirt/top, Pullover, Dress, Coat, and Shirt. Define footwear group F as Sandal, Sneaker, and Ankle boot. Every incorrect prediction is assigned to exactly one bucket: both labels in G; both in F; or every other off-diagonal label pair. This operational taxonomy is disjoint and exhaustive. It describes where errors occur, rather than assigning unobserved causes such as low contrast or occlusion.

Table 10: Exhaustive CNN error groups. Error shares use denominator 818; test-set shares use 10,000. Percentages are rounded independently.

Error group	Count	Share of errors (%)	Share of test set (%)
Within garment group G	678	82.89	6.78
Within footwear group F	79	9.66	0.79
All other confusions	61	7.46	0.61
All CNN errors	818	100.00	8.18

Within-G mistakes account for 82.89% of errors, within-footwear mistakes for 9.66%, and residual confusions for 7.46%. The residual bucket includes cross-group mistakes and errors involving Trouser or Bag. It must be retained: reporting only visually similar tops and footwear would omit 61 measured failures.

12.2 Error Frequency and Common Class Confusions

Table 11 lists the eight largest directed CNN confusion cells. Each true class has support 1,000, so its conditional error rate is the count divided by 1,000. These eight cells contain 485 errors, 59.29% of all errors, and all lie within G.

Table 11: Eight largest directed CNN confusions, ordered by count and then class ID for ties. A direction and its reverse are separate error cells.

True class	Predicted class	Count	True-class rate (%)	Share of all errors (%)
Shirt	T-shirt/top	112	11.2	13.69
Shirt	Coat	82	8.2	10.02
Shirt	Pullover	61	6.1	7.46
T-shirt/top	Shirt	59	5.9	7.21
Pullover	Coat	54	5.4	6.60
Dress	Shirt	43	4.3	5.26
Coat	Pullover	37	3.7	4.52
Coat	Shirt	37	3.7	4.52

Shirt contributes 285 misses in total. Its predictions as T-shirt/top, Coat, and Pullover contribute 112, 82, and 61 cases: together 255, or 89.47% of missed Shirts. The reciprocal T-shirt/top–Shirt pair contains 171 errors, while Coat–Shirt contains 119. Direction matters: Shirt-to-T-shirt/top has 112 cases, versus 59 in reverse. Improvement should therefore be evaluated using both Shirt recall and false-positive errors in its neighboring categories, rather than optimizing one directed cell in isolation.

12.3 Representative Failure Cases

Figure 19 provides one recorded CNN case per taxonomy bucket. Test #40 is Shirt predicted as T-shirt/top; test #68 is Ankle boot predicted as Sneaker; test #32 is Dress predicted as Trouser. The last case is a residual-group error, not a Shirt confusion. Its direction occurs four times in the complete CNN matrix, so this early example should not be mistaken for a dominant failure mode.



Figure 19: One identified CNN failure per error bucket. The cases are recovered from the saved incorrect-example grid and rendered from original test pixels. Group definitions determine the count categories; visible attributes do not establish the model's causal decision process.

The short-sleeved outline in #40 resembles the predicted neighboring category. The boot in #68 shares a closed footwear silhouette with sneakers, while the narrow view of #32 leaves limited category cues. Other examples in Figure 15, such as the checked Pullover #248 predicted as Shirt, show texture-related ambiguity. These are visual hypotheses. No complete dataset of contrast, viewpoint, or texture tags was recorded, so their causal frequencies cannot be inferred from the class-based taxonomy.

12.4 Hypothesized Causes and Improvement Ideas

1. **Overlapping garment appearance.** Test whether modest, label-preserving translations and contrast variation improve validation Shirt recall and macro-F1 without reducing neighboring-class precision. The current transform code accepts no augmentation beyond `none`; this requires a new implementation and controlled experiment. Aggressive crops may remove sleeves or garment length, so an improvement is not assumed.

2. **Detail lost during downsampling.** Compare the CNN with a version retaining more spatial resolution, under a matched parameter or compute budget. Report validation garment F1 and inference cost together. The current images are only 28×28; retaining feature resolution cannot recover detail absent from the raw input.

3. **Footwear boundary cases.** Audit ankle/sole visibility and distinguishable silhouettes on training/validation examples, then test a controlled resolution or pooling change. Use the complete footwear confusion cells as the outcome, not the proportion of shoe images in a selected grid.

4. **Residual and uncertain cases.** Inspect the original pixels and official labels before alleging annotation errors. Save future full per-image predictions and logits to support paired comparisons, confidence/calibration analysis, and a larger reproducible failure sample. These artifacts are absent from the current evaluation outputs.

Any new design should be selected on training/validation data with a stated search budget. Since the test errors have now informed these hypotheses, repeating model selection against this same test set would weaken its independence. The present report claims no measured gain for these proposed remedies.

13 Optional Ablation and Extensions

The available extension is the three-way LSTM representation study. Assignment 1 recommends such a study; additional architecture families or datasets are optional [11]. We use the existing row, column, and patch runs. This section adds no newly trained configuration and does not describe the comparison as a fully isolated causal ablation.

13.1 Hypothesis and Experimental Design

Two questions organize the recorded comparison. First, does changing row to column scan order affect predictive quality at the same recurrent capacity? Second, does shortening the sequence through patches reduce computation, and what predictive cost accompanies that change? These are retrospective analysis questions; no claim of preregistered hypotheses is made.

All three use the fixed Stratified Split, seed 42, batch size 128, Adam at 10^{-3}, a maximum of 50 epochs, patience 5, validation macro-F1 checkpointing, hidden size 128, one unidirectional

LSTM layer, and no explicit recurrent dropout. Rows and columns have the same $[B, 28, 28]$ sequence shape and 82,186 parameters, making scan orientation their intended changed factor. Patches use $[B, 16, 49]$ and 92,938 parameters, changing granularity, recurrent depth in time, and capacity together. Actual completed epochs also differ.

The descriptive decision criterion is joint validation/test quality and measured cost, without a significance threshold inferred after the fact. Accuracy and macro-F1 quantify prediction quality; parameter count, arithmetic estimates, and inference time describe cost. Test scores characterize these fixed runs and are not used here to tune a new model.

13.2 Results

Table 12: Changes relative to LSTM rows. Accuracy and macro-F1 changes are percentage points; parameter and inference changes are relative percentages. Negative inference change means a shorter recorded batched forward time.

Variant	Acc. change (points)	F1 change (points)	Parameter change (%)	Inference change (%)
Columns	-1.31	-1.23	+0.0	+2.9
Patches	-1.59	-1.51	+13.1	-18.4

Rows lead the available variants on accuracy and macro-F1. Patches reduce measured forward time by 18.4% but increase size by 13.1%, while losing 1.59 accuracy points. Figure 11 shows lower patch training loss together with worse late validation loss. The operation decomposition in Section 11 identifies fewer recurrent hidden-state updates as the source of its lower counted arithmetic. Additional column and patch confusion matrices are included in Appendix C.

13.3 Discussion

The results support choosing rows among these recorded LSTM configurations when quality is the priority. They also demonstrate that reducing sequence length can lower one computational cost without improving generalization or model size. A smaller patch size, hidden-size matching, regularization changes, or bidirectional processing would require new runs. No GRU, Mamba, CIFAR-10, pretrained-backbone, augmentation, or cross-framework result is claimed; code support for an option is not an evaluated experiment.

Only one seed is recorded, and both additional runs carry `c1747a1-dirty`. Their exact working-tree changes cannot be reconstructed from the commit identifier alone. These limits prevent treating the observed orientation or patch effects as precise population estimates or attributing every difference to a single factor.

14 Limitations

14.1 Data Limitations

Fashion-MNIST contains isolated grayscale fashion items at 28×28, with ten balanced classes. It supplies neither color nor fine product detail, and the official split is image-based rather than a verified product-, source-, or subject-disjoint split. We checked index separation and training-only normalization, but did not perform a full near-duplicate or provenance audit. Index disjointness alone does not establish independence of every underlying item.

Balanced support prevents class frequency from explaining the observed difficulty differences, but the label categories can still overlap visually. Selected examples do not establish annotation noise, intrinsic class ambiguity rates, or an irreducible error bound. The first-16 correct/incorrect grids also cover only a small, ordered subset of the test outcomes; they are illustrative, not representative samples of all failure attributes.

14.2 Method and Experimental Limitations

Each configuration has one training seed. No seed-level confidence interval, significance test, or estimate of run-to-run variance is available. The architectures differ in capacity, normalization, dropout, and representation; shared optimizer settings do not establish equally good tuning. The recorded experiment includes no learning-rate search, scheduler, warm-up, augmentation study, mixed-precision comparison, or matched-capacity architecture control.

Training and validation losses are measured in different modes and at different stages of an epoch, so their difference is not a controlled generalization gap. Early stopping changes total training work. Saved inference time is a batched, forward-only median on the recorded CUDA environment; it excludes preprocessing and transfers, and the individual timing samples are not retained. Parameter count and analytical MACs do not measure peak memory or application latency.

Reproduction has material limits: original checkpoints are absent from this checkout; two LSTM variants record dirty source states; evaluation revision was not independently stored; and the dependency snapshot differs from run metadata and contains machine-local paths. Saved configurations, split indices, histories, metrics, and source fingerprints remain auditable, but they do not recover missing weights or working-tree changes. Section 16 distinguishes artifact re-analysis from new training.

14.3 Generalization

The conclusions apply to the reported Fashion-MNIST configurations and protocol. No external dataset, real catalog photograph, distribution-shift benchmark, adversarial evaluation, or calibrated deployment threshold was tested. A model leading macro-F1 here may not lead a different workload, class-cost objective, image resolution, or hardware platform. In particular, the CNN's overall lead does not imply higher recall for every class or correctness on every image.

The test set has been inspected for reporting and error analysis. Future hypotheses derived from those observations require validation-based selection and, for a stronger independent confirmation, an appropriate new held-out evaluation. The report provides evidence for recorded outcomes and structural explanations, while controlled reruns are needed for stronger causal and deployment claims.

15 Conclusion

15.1 Main Findings

This study implemented and compared five model families through a shared PyTorch pipeline on Fashion-MNIST, with a fixed 54,000/6,000/10,000 train/validation/test partition and validation macro-F1 checkpoint selection. All seven recorded configurations use seed 42.

The CNN achieves the strongest aggregate test result: 91.82% accuracy and 91.76% macro-F1

with 50,378 parameters. It reduces errors by 244 relative to the MLP and 779 relative to the linear baseline. Its advantage is supported by class-level counts and selected common-image predictions, while Dress recall remains higher for the row LSTM. Shirt is the weakest-F1 class across all five main models.

The main research question therefore has a conditional answer: CNN offers the best recorded predictive quality with compact parameter size, whereas Linear and MLP retain lower-latency operating points under the measured batched protocol. Rows give the strongest LSTM quality among the tested representations. Patches lower recurrent arithmetic and inference cost while worsening test quality; the Transformer supplies the largest configured model without a corresponding gain under the shared recipe.

These observations are consistent with a useful spatial prior on small grayscale images, but do not isolate architecture from regularization, optimization, or capacity. One seed, missing original checkpoints, incomplete dirty-run traceability, and environment differences limit stronger conclusions.

15.2 Future Work

The next experimental priorities are repeated seeds, complete prediction/checkpoint exports, and controlled validation studies. Match capacity and normalization when comparing dense and convolutional models; vary LSTM orientation or patch size with explicit capacity controls; and tune the patch encoder's optimization and regularization under a comparable search budget. Test candidate changes on validation data and evaluate the selected design independently after freezing the protocol. These steps would distinguish reproducible architectural effects from configuration-specific outcomes.

16 Reproducibility

Two reproduction tasks are distinguished: recomputing tables/plots from archived artifacts, which does not require weights, and training/evaluating a new model, which does. The report-generation additions accompany this working copy; the remote source snapshot alone does not contain these uncommitted report additions until they are published by the group.

16.1 Repository and Commit Reference

The repository is `https://github.com/nguyenhohongphuc/co3133-deep-learning-project`. The inspected core-code checkout is `a39863859c1f8676ee85687eb230b54fd617cd75`. Five main runs record training commit `c1747a1260a9745d8da341b6fab3d64546538488`; the column and patch LSTMs append `-dirty` to its short form.

A source comparison between these revisions shows that only `src/evaluate.py` differs within `src/` and `configs/`. Model, data, seed, training-loop, and configuration sources in the inspected checkout therefore match the committed training version. This does not recover the two uncommitted training states. Evaluation metadata also omit their own source revision, so the current evaluator is fingerprinted separately rather than assigned an undocumented historical commit.

16.2 Environment and Dependencies

The recorded experiments use Python 3.10.21, PyTorch `2.14.1+cu126`, CUDA 12.6, and an RTX 4060 Laptop GPU. Section 7 separates these captured fields from README hardware descriptions. Recreating a training environment requires a compatible PyTorch/CUDA build and the repository dependencies: torchvision, NumPy, pandas, matplotlib, scikit-learn, PyYAML, and tqdm.

`requirements.txt` is an unpinned dependency list. `requirements.lock.txt` is evidence of one exported environment, not a portable exact installer: it contains local file URLs and lists torch 2.14.0 rather than the recorded build. Reconcile these differences before claiming an exact environment recreation. A basic environment check is:

```
python -c "import sys, torch; print(sys.version); \
print(torch.__version__, torch.version.cuda); \
print(torch.cuda.is_available())"
```

The seed helper seeds Python, NumPy, PyTorch, CUDA, and workers; it enables deterministic cuDNN behavior and disables cuDNN benchmarking. It does not establish identical numerical results across arbitrary devices or dependency versions. Report rendering additionally uses matplotlib/Pillow and Tesseract 5.5.1 to audit existing annotation text, not to infer fashion labels.

16.3 Dataset Preparation and Split Reconstruction

From the repository root, the dataset downloader is:

```
python scripts/prepare_data.py \
  --dataset fashion-mnist --out data/
```

It obtains both official partitions. Reuse `results/a1/splits/split_seed42.json`: it contains exact training/validation indices and class counts. The official test sequence remains unchanged. If rebuilding in an isolated reproduction copy, the supported split command is:

```
python -m src.data.split --config configs/a1.yaml
python scripts/compute_a1_stats.py
```

The first command overwrites the configured split file, so verify the regenerated indices against the archived file before replacing evidence. Normalization must be computed from the 54,000 training images only; the expected rounded values are 0.286209 and 0.353160. The input manifest records the archived split's SHA-256 and source-file hashes.

16.4 Training and Evaluation Commands

The current trainer supports `--model`; it reads `configs/a1.yaml` and does not accept `--config` or `--seed`. For a direct run, the supported commands are:

```
python -m src.train --model linear
python -m src.train --model mlp
python -m src.train --model cnn
python -m src.train --model rnn
python -m src.train --model transformer
```

Each uses the active central configuration, so these five commands alone do not select archived variants. For an archived setting, the supplied helper loads that run's saved configuration and prints a read-only plan by default:

```
python scripts/a1_report/reproduce_a1.py \
  --run-id a1_cnn_s42_1027-0157
```

In an isolated copy with the intended environment and data prepared, explicit execution is:

```
python scripts/a1_report/reproduce_a1.py \
  --run-id a1_cnn_s42_1027-0157 --execute
```

The helper temporarily applies the snapshot, trains into a new run directory, identifies that directory, evaluates it through `src.evaluate --run_dir`, and restores the central configuration in a `finally` block. It never substitutes new weights or metrics into the archived run. Copying a different configuration can legitimately produce a dirty new-run identifier; the new snapshot and actual source revision must remain recorded. Use the other IDs in Table 13 to select their settings. The helper's plans were checked during report preparation; its training/evaluation execution path was not run for this manuscript.

16.5 Checkpoint Access

No `best.pt` file is present in the inspected A1 run folders, and no verified checkpoint-download location is supplied in the repository. The `*.pt` ignore rule explains their absence from version control but does not provide the weights. A supplied original checkpoint would belong at `results/a1/runs/<run_id>/best.pt`; otherwise use the reconstruction procedure above and retain the newly generated run ID.

Historical confusion matrices and example grids can be audited without weights. They cannot recreate full logits, predictions, activations, or the original model state. Reconstructed models may differ numerically, and exact reconstruction of dirty variants remains limited by missing working-tree changes. These limits are disclosed rather than replaced by an unverified download link.

16.6 Experiment Traceability

Table 13: Archived experiment identities. All seeds are 42; best indices are zero-based. SHA-256 prefixes identify the saved configuration bytes; full hashes appear in the input manifest.

Run ID (seed 42)	Best index	Training commit	Config SHA-256 prefix
a1_linear_s42_1027-0134	11	c1747a1	e8ee028d46
a1_mlp_s42_1027-0143	22	c1747a1	e8ee028d46
a1_cnn_s42_1027-0157	7	c1747a1	e8ee028d46
a1_rnn_rows_s42_1027-0204	21	c1747a1	e8ee028d46
a1_transformer_patches_s42_1027-0242	11	c1747a1	e8ee028d46
a1_rnn_cols_s42_1027-0217	20	c1747a1-dirty	f5cce79353
a1_rnn_patches_s42_1027-0229	20	c1747a1-dirty	41df42c30d

Full source hashes and weight availability are recorded in `results/a1/tables/a1_reproducibility_manifest.json`. The report README documents figure regeneration and export as `NNs_A1_Report.pdf`.

AI Usage Disclosure

Generative AI was used in implementation and report preparation. The detailed, stage-specific record is `AI_USAGE.md`; these summaries cover A1 use, responsible members, and verification limits. No new predictive experiment was performed during report completion. No synthetic fashion image, invented result, or fabricated checkpoint link is presented as evidence.

Claude: Implementation and Early Documentation

The existing A1 log records Claude Code, model Opus 5, used by Hồ Hồng Phúc Nguyên (2352824) on 22–23 September 2026. Its purposes were data-pipeline review, metrics and training/evaluation implementation, debugging, concept explanation, and initial documentation. Affected areas include data modules, the training engine, CNN/pipeline entry points, plotting, configuration, and website/README setup descriptions.

Representative requests recorded in the log include "write metrics.py and wire up the checkpointing" and reviewing a teammate's data-pipeline changes. The implementation record describes metric checks against scikit-learn, split/normalization checks, and end-to-end development runs. It also records corrected AI-assisted defects involving missing imports, timing on short loaders, and Windows console encoding. These are historical verification claims attributed to that log, rather than tests newly repeated for this manuscript. Sources include PyTorch/scikit-learn documentation, the handbook, code, and saved runs. Hồ Hồng Phúc Nguyên is the responsible verifier named for those entries.

OpenAI Codex: Report Drafting and Evidence Analysis

OpenAI Codex was used by Nguyễn Nhật Nam (2352778) on 27–28 October 2026; the model identifier was not recorded. Assistance covered Sections 1–16, the abstract, appendices, bibliography, diagrams, tables, artifact audits, and reproduction documentation. Representative requests, translated, were to write the report section by section, compare models with evidence from saved figures, and complete the manuscript for one review.

The output was checked against implemented models, all seven saved configurations, split indices and raw labels, training-only normalization, histories, and test matrices. Parameter and synthetic forward-shape checks are distinguished from predictive evaluation. Accuracy/classwise metrics were recomputed from confusion counts; checkpoint and stopping rules were checked against histories; common-image examples were tied to original test IDs. Architectural explanations were compared with primary papers and official documentation. Reproducible scripts retain figures, hashes, operation estimates, and derived statistics.

Supporting Tools, Scope, and Responsibility

Local Tesseract 5.5.1 read text already printed in result grids. It did not classify fashion images or generate model predictions. Pixel matching and ground-truth checks identify examples, while saved annotations supply predicted labels. Matplotlib generated scientific plots from recorded data; schematic pipeline/architecture illustrations are distinguished from measured activations or attribution maps.

The report preserves missing-checkpoint, dirty-source, single-seed, and environment limitations. Proposed experiments have no attributed performance gain. No unrecorded AI use is attributed to other members. Appendix D and `AI_USAGE.md` retain contribution evidence and the detailed

log for the group's final review.

References

[1] Zalando Research. *Fashion-MNIST: Dataset repository and documentation*. https://github.com/zalandoresearch/fashion-mnist. Accessed 27 October 2026.

[2] Zalando SE. *Fashion-MNIST MIT License*, 2017. https://github.com/zalandoresearch/fashion-mnist/blob/master/LICENSE. Accessed 27 October 2026.

[3] PyTorch contributors. *LSTM: PyTorch 2.14 documentation*. https://docs.pytorch.org/docs/2.14/generated/torch.nn.LSTM.html. Accessed 27 October 2026.

[4] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin. *Attention Is All You Need*, 2017. https://arxiv.org/abs/1706.03762.

[5] PyTorch contributors. *TransformerEncoderLayer: PyTorch 2.14 documentation*. https://docs.pytorch.org/docs/2.14/generated/torch.nn.TransformerEncoderLayer.html. Accessed 27 October 2026.

[6] PyTorch contributors. *CrossEntropyLoss: PyTorch 2.14 documentation*. https://docs.pytorch.org/docs/2.14/generated/torch.nn.CrossEntropyLoss.html. Accessed 28 October 2026.

[7] PyTorch contributors. *Adam: PyTorch 2.14 documentation*. https://docs.pytorch.org/docs/2.14/generated/torch.optim.Adam.html. Accessed 28 October 2026.

[8] Scikit-learn contributors. *f1_score: scikit-learn 1.7.2 documentation*. https://scikit-learn.org/1.7/modules/generated/sklearn.metrics.f1_score.html. Accessed 28 October 2026.

[9] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner. *Gradient-Based Learning Applied to Document Recognition*. Proceedings of the IEEE, 86(11):2278–2324, 1998. https://leon.bottou.org/papers/lecun-98h.

[10] A. Dosovitskiy et al. *An Image Is Worth 16x16 Words: Transformers for Image Recognition at Scale*. ICLR, 2021. https://arxiv.org/abs/2010.11929.

[11] CO3133, Deep Learning and Its Applications. *Course Project Handbook, Semester 261*. Revision 14 September 2026, Sections 3–5 and 8–14. Local repository copy: `handbook-ene-v2.pdf`.

A Complete Hyperparameter Table

The saved per-run YAML files are the complete machine-readable configurations. The tables summarize their active settings; unused blocks for other models in a snapshot do not define additional experiments. All runs use seed 42, the fixed Stratified Split, training-only normalization, and no augmentation.

Table 14: Shared training settings and implementation defaults in the recorded A1 runs.

Setting	Value
Partitions	54,000 train; 6,000 validation; 10,000 official test
Normalization	Mean 0.286209; standard deviation 0.353160
Batch size / workers	128 / 2; training shuffle only
Training objective	Mean cross-entropy on logits; sample-weighted epoch means
Optimizer	Adam; base learning rate 0.001; weight decay 0
Adam defaults	$\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$
Epoch budget / patience	Maximum 50; patience 5
Monitor / checkpoint	Strictly highest validation macro-F1
Scheduler / warm-up	None / none
Augmentation / label weighting	None / none
Gradient clipping / accumulation	None / none
Mixed precision	Not used
Training device	CUDA; RTX 4060 Laptop GPU in run metadata
Inference timing	Batch 128; 3 warm-up calls; median of 5 full passes

Table 15: Architecture settings. Defaults such as normalization parameters were checked in the instantiated implementations; YAML snapshots retain explicitly configured values.

Configuration	Active architecture settings
Linear	Flatten 784 pixels; affine $784 \to 10$; bias enabled
MLP	$784 \to 256 \to 128 \to 10$; ReLU; dropout 0.2 after hidden activations; biases enabled
CNN	Channels 32, 64; 3×3 kernels, stride 1, padding 1; BatchNorm, ReLU, 2×2 max pooling in each block; head dropout 0.25; biases enabled
CNN BatchNorm	Epsilon 10^{-5}; momentum 0.1; affine parameters and running statistics enabled
LSTM rows / columns	28 steps, width 28; hidden size 128; one layer; unidirectional; no explicit dropout
LSTM patches	Non-overlapping 7×7 patches; 16 steps, width 49; same recurrent settings as rows
Transformer	16 patches of 7×7 plus CLS; width 128; 4 heads; 4 encoder blocks; feed-forward width 256; GELU; dropout 0.1; learned positional embeddings
Transformer head	Post-normalized blocks; final CLS LayerNorm and $128 \to 10$ classifier; LayerNorm epsilon 10^{-5}

B Detailed Architecture Tables

Table 16 lists parameter-bearing components. Zero-parameter reshapes, activations, pooling, and dropout follow Section 5. BatchNorm running statistics are buffers and are excluded. All models return $[B, 10]$ logits; local synthetic shape checks are recorded in `a1_model_verification.json` and are not checkpoint inference.

Table 16: Parameter-bearing components of the implemented models.

Model	Component	Parameters
Linear	Affine $784 \to 10$	7,850
MLP	Affine $784 \to 256$	200,960
	Affine $256 \to 128$	32,896
	Affine $128 \to 10$	1,290
	Total	235,146
CNN	First convolution / BatchNorm	320 / 64
	Second convolution / BatchNorm	18,496 / 128
	Affine $3136 \to 10$	31,370
	Total	50,378
LSTM rows/cols	Gate input/recurrent matrices and biases	80,896
	Affine $128 \to 10$	1,290
	Total	82,186
LSTM patches	Gate input/recurrent matrices and biases	91,648
	Affine $128 \to 10$	1,290
	Total	92,938
Transformer	Patch projection $49 \to 128$	6,400
	CLS / learned positions	128 / 2,176
	Four encoder blocks ($4 \times 132{,}480$)	529,920
	Final LayerNorm / affine classifier	256 / 1,290
	Total	540,170

C Additional Curves and Confusion Matrices

The main loss curves and validation macro-F1 appear in Figures 9 and 10; Figure 11 covers all three recurrent histories. Figure 12 gives the five main matrices. Figure 20 completes coverage of all seven configurations.

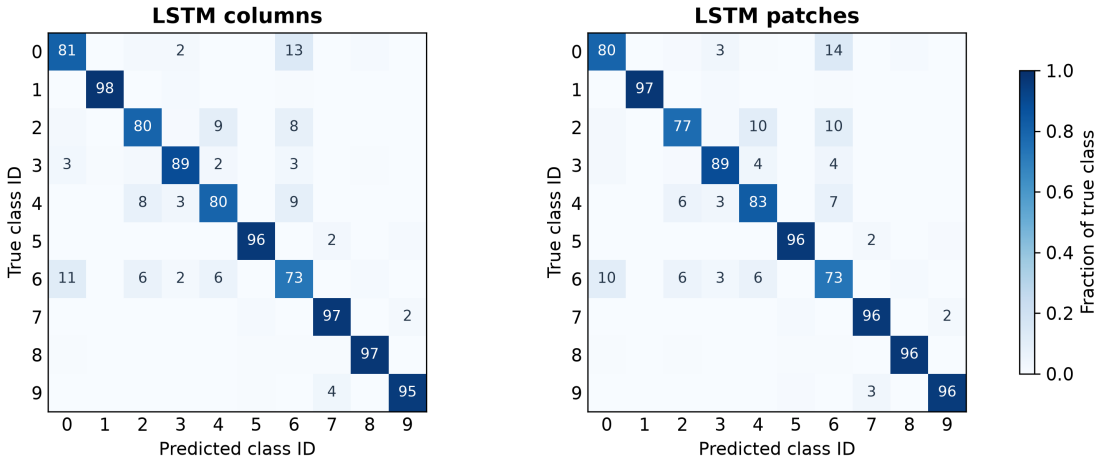


Figure 20: Additional LSTM test confusion matrices, row-normalized by 1,000 images per class. IDs follow Figure 12. Labels show rounded percentages on the diagonal and off-diagonal rates of at least 2%; unlabeled cells may contain errors. Both runs use seed 42 and record dirty source states.

Full integer matrices remain in each run's `test_metrics.json`; all 70 model/class precision, recall, F1, and support records are in `results/a1/tables/a1_classwise_metrics.csv`. Original `curves.png`, `confusion_test.png`, `correct_test.png`, and `wrong_test.png` remain under each run's `figures/`. The report reuses or replots these artifacts without generating new predictions.

D Member Contributions

This record summarizes documented repository work, rather than assigning unrecorded hours or contribution percentages. Commit authorship and the AI log support the scopes below; they do not establish exclusive ownership of every line or record all collaborative activity.

Table 17: Documented contributions and evidence. Names and student IDs follow the group record.

Member	Documented work and evidence
Nguyễn Nhật Nam 2352778	Fashion-MNIST pipeline/EDA (`f760d50`); collaborative LaTeX drafting, figures, and artifact analysis with Codex, as recorded in `AI_USAGE.md`. Responsible for the requested final manuscript review.
Hồ Hồng Phúc Nguyên 2352824	Shared training/evaluation framework, Linear, MLP and CNN (`d93bcd5`), result aggregation, and documentation, supported by ownership comments, `AI_USAGE.md` A1 Entries 1–3, and commits `6984a3e` and `fbf5cdc`.
Phan Quốc Đại Sơn 2353053	Recurrent and Transformer implementations and representation experiments, supported by `36bdc2e` and integration in `885f14e`. Later shared fixes retain their own commit authors.

Final human verification of this contribution record remains pending the group's review.